Lerneinheit 4 · pandas und Matplotlib: der Datenworkflow

M2 · Data-Science-Werkzeugkasten

Version 1.0

Lernziele

Nach dieser Lerneinheit können Sie …

- ☐ eine CSV-Datei in einen DataFrame einlesen und die Datentypen und fehlenden Werte prüfen,
- ☐ erklären, woraus eine Spalte eines DataFrames besteht und wie sie zum NumPy-Array wird,
- ☐ Zeilen und Spalten mit Masken, `loc` und `iloc` auswählen und den Unterschied zwischen Zuordnung über den Index und über die Position erklären,
- ☐ einen Datensatz nach einem festgelegten Vertrag bereinigen und die Bereinigung mit einer Prüfzelle nachweisen,
- ☐ Werte je Gruppe zusammenfassen und die Wirkung fehlender Werte auf Kennzahlen benennen,
- ☐ ein Ergebnis mit Matplotlib als beschriftetes Balken-, Histogramm- oder Streudiagramm darstellen,
- ☐ aus festgelegten Merkmalsspalten ein Array der Form (N, D) mit dem Datentyp `float32` und die Zielspalte getrennt davon erzeugen.

In den ersten drei Lerneinheiten waren die Daten Vektoren und Tensoren: Zahlenblöcke gleicher Art, auf denen NumPy rechnet. Die meisten Daten, mit denen ein Modell später trainiert wird, sehen anders aus. Sie kommen als Tabelle, mit einer Zeile je Beobachtung und Spalten verschiedener Art: Zahlen, Text, Kategorien. Dazu kommen Probleme mit der Datenqualität, etwa fehlende Werte, falsche Werte und uneinheitliche Schreibweisen. Bevor aus einer solchen Tabelle ein Tensor wird, muss sie eingelesen, geprüft, bereinigt und verstanden werden. Diesen Ablauf nennen wir Datenworkflow. pandas ist eine der Standardbibliotheken dafür, Matplotlib stellt die Ergebnisse grafisch dar.

Mit dieser Lerneinheit beginnt eine Folge von drei Lerneinheiten, die mit dem Training eines kleinen Modells endet: In dieser Lerneinheit entsteht aus einer unsauberen Datei eine bereinigte Tabelle, in der nächsten wird daraus die Eingabe für ein Modell, und danach trainieren wir damit ein neuronales Netz. Der Datensatz dafür ist die Passagierliste der Titanic, ein verbreiteter Lehrdatensatz. Er enthält Zahlen und Kategorien, fehlende Werte und eine Zielspalte, die sich vorhersagen lässt.

Was aus LE03 weitergilt: Ein DataFrame ist eine Sammlung benannter Spalten, und was LE03 über Masken, Achsen und Reduktionen gezeigt hat, gilt hier weiter, nur dass Spalten jetzt Namen haben. Eine Maske wählt Zeilen aus, `groupby` fasst je Gruppe zusammen, und `to_numpy()` ist der Weg zurück zum Array. Neu ist ein Unterschied: pandas ordnet über den Index zu, NumPy über die Position.

Inhalt

- 4.1 Von Listen zu Tabellen
- 4.2 Der Datensatz und das Einlesen
- 4.3 Auswählen und Filtern
- 4.4 Bereinigen mit Vertrag
- 4.5 Gruppieren und Zusammenfassen
- 4.6 Darstellen mit Matplotlib
- 4.7 Von der Tabelle zum Array

4.1 Von Listen zu Tabellen

Die 400 Absätze aus den Kursdaten kennen Sie als Liste von Dicts: jeder Absatz ein Dict mit `id`, `title` und `text`. In Grundkurs Teil 2 haben Sie darauf von Hand gezählt, zum Beispiel die Absätze je Titel. pandas macht aus einer solchen Liste eine Tabelle, und die meisten dieser Fragen lassen sich damit in einer Zeile Code beantworten. Die nächsten beiden Zellen stellen die Daten bereit und richten die Bibliotheken ein, führen Sie beide einfach aus. Die zweite prüft außerdem die pandas-Version: Dieses Notebook ist für pandas 3 geschrieben. Auf einer älteren Umgebung schaltet die Zelle das Verhalten ein, das pandas 3 zum Standard macht (Copy-on-Write, erklärt im Kapitel 4.4), und sagt, was dann anders aussieht.



Eine Liste von Dicts ist der Zettelstapel: Jede Frage heißt alle Zettel durchgehen. Ein DataFrame ist der Karteikasten mit beschrifteten Fächern: Spalten sind direkt ansprechbar, und Fragen wie Filtern oder Zählen sind ein einzelner Aufruf statt einer eigenen Schleife.

```
# Vorgabe: Kursdaten bereitstellen — einfach ausführen
# Die Zelle legt den Ordner data/ an und lädt fehlende Dateien aus dem Kursrepo nach:
#   data/appetizer_passages.json
#   data/titanic_raw.csv
#   data/DATENBLATT_titanic.md
```

```
   # Vorgabe: Bibliotheken und Kursdaten – einfach ausführen
   import json, warnings
   import numpy as np
① import pandas as pd
   import matplotlib.pyplot as plt

   PANDAS_3 = int(pd.__version__.split(".")[0]) >= 3
   if not PANDAS_3:
       pd.options.mode.copy_on_write = True      # ältere pandas-Version: das Verhalten von pandas 3
einschalten
       print(f"pandas {pd.__version__}: Copy-on-Write eingeschaltet; Textspalten erscheinen hier als object
statt str")
   else:
       print("pandas", pd.__version__)

   with open("data/appetizer_passages.json", encoding="utf-8") as f:
       passages = json.load(f)["passages"]       # 400 Absätze als Liste von Dicts
   print(len(passages), "Absätze, der erste:", {k: str(v)[:30] for k, v in passages[0].items()})
```

① `pd` und `plt` sind die üblichen Kurznamen für pandas und das Zeichenmodul von Matplotlib

```
pandas 3.0.5
400 Absätze, der erste: {'id': '0', 'title': 'Aufzugsanlage', 'text': 'Seilloser Aufzug.\nAn der RWTH
'}
```

`pd.DataFrame(liste_von_dicts)` baut aus den Dicts eine Tabelle: Jeder Schlüssel wird eine Spalte, jedes Dict eine Zeile. `head` zeigt die ersten Zeilen, zur Kontrolle, ob die Tabelle so aussieht wie erwartet. Die Attribute daneben beschreiben die Tabelle so, wie `shape` und `dtype` in LE03 ein Array beschrieben haben.

```
① df_p = pd.DataFrame(passages)
   print(df_p.head(3))                # die ersten drei Zeilen
   print()
   print("shape:", df_p.shape)          # (Zeilen, Spalten), wie bei einem Array
   print("columns:", list(df_p.columns))
② print(df_p.dtypes)
```

① `pd.DataFrame(...)` nimmt eine Liste von Dicts; die Schlüssel werden Spaltennamen

② `dtypes` nennt je Spalte den Datentyp: `int64` für die Nummern, `str` für die Texte (unter pandas 3); die letzte Zeile `dtype: object` gehört zur Ausgabe selbst: `dtypes` ist eine Series aus Datentypen, und deren eigener Datentyp ist `object`

```
   id          title                                               text
0   0  Aufzugsanlage  Seilloser Aufzug.\nAn der RWTH Aachen im Insti...
1   1        Sichuan  Landwirtschaft.\nSichuan gilt vorwiegend als R...
2   2        Antenne  Polarisation.\nAntennen strahlen polarisierte ...

shape: (400, 3)
columns: ['id', 'title', 'text']
id       int64
title      str
text       str
dtype: object
```

Für alles, was sich im Kopf nachvollziehen lassen soll, gibt es ab hier eine kleine Tabelle `small` mit fünf Passagieren und drei Spalten; die Grafik unten zeigt dieselben Werte. Die Kursdaten `df_p` und später die Passagierliste bleiben daneben für die echten Fragen.

```
① small = pd.DataFrame({"pclass": [1, 1, 3, 3, 2],
                        "sex": ["female", "male", "male", "female", "male"],
                        "age": [29.0, 0.75, np.nan, 18.0, 54.0]})
  print(small)
```

① ein Dict aus Spaltenname -> Liste ist die zweite übliche Form, einen DataFrame anzulegen; `np.nan` ist ein fehlender Wert

```
   pclass     sex    age
0       1  female  29.00
1       1    male   0.75
2       3    male    NaN
3       3  female  18.00
4       2    male  54.00
```

Ein DataFrame ist eine Sammlung benannter Spalten. Jede Spalte ist eine `Series`: eine Folge von Werten mit genau einem Datentyp. Dazu kommt der Index: die Liste der Zeilenetiketten, ein Etikett je Zeile. Alle Spalten eines DataFrames teilen sich diesen einen Index, deshalb gehört der dritte Eintrag von `pclass` zur selben Zeile wie der dritte Eintrag von `age`.

Standardmäßig sind die Etiketten die Nummern 0 bis N minus 1, die `pd.DataFrame` beim Anlegen vergibt. Es können aber auch Namen oder Datumsangaben sein, und nach einem Filter behalten die übrigen Zeilen ihre alten Nummern. Deshalb heißt es Etikett und nicht Position: Das Etikett bleibt an der Zeile, die Position zählt im Ergebnis neu von 0. In NumPy gibt es nur die Position.

Die Werte einer Zahlenspalte liegen als NumPy-Array im Speicher, und `to_numpy()` gibt dieses Array zurück. Textspalten speichert pandas 3 in einem eigenen String-Datentyp; `to_numpy()` stellt auch sie als NumPy-Array bereit. Was Sie über Arrays wissen, gilt deshalb spaltenweise weiter: ein Datentyp je Spalte, Rechenoperationen wirken auf alle Einträge zugleich, und eine Reduktion wie `sum` oder `mean` fasst die Spalte zu einer Zahl zusammen. Das Kapitel 4.3 zeigt, was aus Etikett und Position folgt. Die Grafik zeigt die Tabelle `small`; ihre drei Spalten nehmen den Datensatz aus Kapitel 4.2 vorweg.

Ein DataFrame ist eine Sammlung benannter Spalten

df

	pclass	sex	age
0	1	female	29.0
1	1	male	0.75
2	3	male	NaN
3	3	female	18.0
4	2	male	54.0

Index: ein Etikett je Zeile

df["age"] →

Series: eine Spalte

Werte	29.0 | 0.75 | NaN | 18.0 | 54.0
dtype	float64
index	0, 1, 2, 3, 4

`to_numpy()` -> Array (5,), float64

Zahlen als NumPy-Array, Text in eigenem Format

Jede Spalte hat genau einen Datentyp. Zeilen spricht man über den Index an, Spalten über ihren Namen.

💭 Kurz überlegen

`df_p["title"]` und `df_p[["title"]]` sehen fast gleich aus. Was ist der Unterschied im Ergebnis?

```
  print(type(df_p["title"]).__name__, df_p["title"].shape)       # eine Series, Form (400,)
  print(type(df_p[["title"]]).__name__, df_p[["title"]].shape)   # ein DataFrame, Form (400, 1)
```

```
Series (400,)
DataFrame (400, 1)
```

✏️ Aufgabe 4.1 #1 Kurzübung

Was kommt heraus? Die nächste Zelle enthält fünf Zeilen auf der Tabelle `small`. Schreiben Sie für jede Zeile Ihre Vorhersage hinter das `# ->` (Werte, Form oder Typ, so genau Sie können), dann führen Sie die Zelle aus und vergleichen. Wo die Vorhersage danebenlag, ändern Sie die Zeile ab und probieren weiter; dafür ist die Zelle da.

```
print(small.shape)                    # -> ?
print(small["age"])                   # -> ?
print(small[["pclass", "age"]].shape) # -> ?
print(small["sex"].value_counts())    # -> ?
print(small["pclass"].sum())          # -> ?
```

Der Unterschied in einem Satz: Eine `Series` ist eine einzelne Spalte, eindimensional wie ein Vektor, mit einem Datentyp und dem Index. Ein DataFrame ist die Tabelle aus solchen Spalten, zweidimensional, mit einem Datentyp je Spalte und einem Index für alle. Spaltenmethoden wie `value_counts`, `mean` oder `.str.len()` rufen Sie auf einer Series auf, `head`, `shape` und `dtypes` gibt es für beide.

Die nächste Zelle beantwortet drei Fragen aus Grundkurs Teil 2 in je einer Zeile. `value_counts` zählt, wie oft jeder Wert in einer Spalte vorkommt, und sortiert dabei absteigend. Das ist die Zuordnung von Titeln zu Absätzen, die Sie dort mit einem Dict und einer Schleife gebaut haben. Über `.str` erreichen Sie die Textmethoden einer Spalte: `df_p["text"].str.len()` wendet `len` auf jede Zelle an und liefert die Textlänge je Absatz als neue Spalte, ohne Schleife. `describe` fasst eine Zahlenspalte mit Anzahl, Mittelwert, Streuung und Quantilen zusammen.

```
① counts = df_p["title"].value_counts()
② print(counts.head(5))
  print("verschiedene Titel:", len(counts))
③ lengths = df_p["text"].str.len()
④ print(lengths.describe().round(1))
⑤ arr = lengths.to_numpy()
  print(type(arr).__name__, arr.shape, arr.dtype, "Mittelwert mit NumPy:", arr.mean().round(1))
```

① `value_counts()` liefert eine Series: Index = die Werte, Einträge = ihre Häufigkeit, absteigend sortiert
② die Ausgabe einer Series endet mit `Name:` und `dtype:`, dem Namen der Series (hier `count`) und ihrem Datentyp; darüber steht `title`, der Name ihres Index
③ `.str.len()` wendet `len` auf jede Zelle der Textspalte an und liefert eine Zahlenspalte
④ `describe()` fasst eine Zahlenspalte mit Anzahl, Mittelwert, Standardabweichung, Minimum, Quartilen und Maximum zusammen
⑤ `to_numpy()` gibt die Werte der Spalte als NumPy-Array zurück, wie Sie es aus LE03 kennen

```
title
Mexiko-Stadt         6
Aufklärung           6
Iran                 5
Bern                 5
Osmanisches Reich    5
Name: count, dtype: int64
verschiedene Titel: 217
count     400.0
mean     1103.5
std       307.5
min       495.0
25%       822.8
50%      1197.5
75%      1378.5
max      1490.0
Name: text, dtype: float64
ndarray (400,) int64 Mittelwert mit NumPy: 1103.5
```

✏️ Aufgabe 4.1 #2 3 min

Schreiben Sie die Funktion `frequent_titles(df, n)`. Sie bekommt den DataFrame der Absätze und eine Zahl `n` und gibt die Liste aller Titel zurück, die mindestens `n` Absätze haben, alphabetisch sortiert. Für `n = 5` sind das neun Titel. Die Prüfzelle vergleicht die Liste. Wie in den früheren Lerneinheiten steht in der Aufgabenzelle der Platzhalter `...` dort, wo Ihr Code hinkommt.

Hinweis: `counts[counts >= n]` wählt aus der Series `counts` die Einträge ab `n` aus, so wie eine Maske in LE03 aus einem Array auswählt. Die Titel stehen dann im Index: `.index` liefert sie, `sorted(...)` ordnet sie.

```
def frequent_titles(df, n):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
from collections import Counter
result = frequent_titles(df_p, 5)
assert result is not None, "frequent_titles gibt noch nichts zurück (return vergessen?)"
assert isinstance(result, list), "das Ergebnis soll eine Liste sein"
assert len(result) == 9, f"für n = 5 sind es 9 Titel, nicht {len(result)}"
assert result == sorted(result), "die Liste soll alphabetisch sortiert sein"
counted = Counter(p["title"] for p in passages)                    # Vergleichswert ohne pandas, wie in Grundkurs Teil 2
assert result == sorted(t for t, c in counted.items() if c >= 5), "die Liste stimmt nicht mit der Zählung über die Dicts überein"
assert frequent_titles(df_p, 6) == ["Aufklärung", "Mexiko-Stadt"], "für n = 6 bleiben zwei Titel"
tiny = pd.DataFrame({"title": ["B", "A", "B", "A", "B", "C"]})       # zweite Tabelle: die Funktion muss df verwenden, nicht df_p
assert frequent_titles(tiny, 2) == ["A", "B"] and frequent_titles(tiny, 3) == ["B"], "die Funktion soll mit der übergebenen Tabelle df rechnen"
print("n = 5:", result)
print("n = 6:", frequent_titles(df_p, 6))
print("Alle Prüfungen bestanden ✓")
```

4.2 Der Datensatz und das Einlesen

Für die nächsten Lerneinheiten brauchen wir einen anderen Datensatz als die Suchdaten: eine Tabelle mit Zahlen, Kategorien und Lücken und mit einer Zielspalte, die ein Modell vorhersagen kann. Der Titanic-Datensatz ist dafür ein klassischer Lehrdatensatz: eine Zeile je Passagier mit Klasse, Geschlecht, Alter, Fahrpreis und weiteren Angaben, dazu die Spalte `survived` mit 1 für überlebt und 0 für nicht überlebt. Er stammt von der Vanderbilt University (Harrell und Cason, zusammengestellt aus der Encyclopedia Titanica), liegt auf OpenML, einer offenen Plattform für Lehr- und Forschungsdatensätze, unter der Nummer 40945 und ist dort als „Public" freigegeben. Das Beispiel „Column Transformer with Mixed Types" aus der scikit-learn-Dokumentation, das die nächste Lerneinheit als Vorlage nimmt, verwendet denselben Datensatz (Link im Quellenblock).

Die Kursfassung `titanic_raw.csv` enthält dieselben Daten mit absichtlich eingebauten Problemen, wie sie in echten Daten vorkommen. Welche das sind, steht im Datenblatt `DATENBLATT_titanic.md`, das die Ladezelle zusammen mit der CSV-Datei nach `data/` legt. Die folgende Übersicht nennt zu jeder Spalte ihre Bedeutung, die Art ihrer Werte und ihre Rolle für das spätere Modell:

Überblick: Die Spalten des Datensatzes

Spalte	Bedeutung	Art	Rolle
`pclass`	Klasse der Fahrkarte, 1 bis 3	Kategorie als Zahl	Merkmal
`survived`	1 überlebt, 0 nicht überlebt	Zahl 0/1	Zielgröße
`name`	Name mit Anrede	Text	bleibt, kein Merkmal
`sex`	Geschlecht, `female` oder `male`	Kategorie	Merkmal
`age`	Alter in Jahren, bei Kindern mit Nachkommastellen	Zahl, Lücken	Merkmal
`sibsp`	Anzahl Geschwister und Ehepartner an Bord	Zahl	Merkmal
`parch`	Anzahl Eltern und Kinder an Bord	Zahl	Merkmal
`ticket`	Fahrkartennummer	Text	bleibt, kein Merkmal
`fare`	Fahrpreis in Pfund	Zahl, Lücken	Merkmal
`cabin`	Kabinennummer	Text, viele Lücken	bleibt, kein Merkmal
`embarked`	Hafen: `C` Cherbourg, `Q` Queenstown, `S` Southampton	Kategorie, Lücken	Merkmal
`boat`	Nummer des Rettungsboots	Text, Lücken	wird entfernt (Leakage)
`body`	Nummer bei Bergung der Leiche	Zahl, Lücken	wird entfernt (Leakage)
`home.dest`	Wohnort und Reiseziel	Text, Lücken	wird entfernt

Merkmale sind die Spalten, aus denen ein Modell später die Zielgröße vorhersagen soll; die Zielgröße ist, was vorhergesagt wird. „Lücken" heißt: In manchen Zeilen fehlt der Wert. `boat` und `body` entstehen erst nach dem Unglück, mit der Rettung und der Bergung, und verraten das Ergebnis fast vollständig: Wer in einem Rettungsboot saß, hat in der Regel überlebt, wessen Leiche geborgen wurde, nicht. Zu dem Zeitpunkt, für den ein Modell vorhersagen soll, gibt es diese Angaben noch nicht. Ein Modell, das sie im Training sieht, lernt die Antwort auswendig statt den Zusammenhang mit Klasse, Alter und Geschlecht, und beim Einsatz fehlen sie. Das heißt Leakage, deshalb werden beide Spalten im Bereinigungskapitel entfernt. (Ausnahmslos ist die Regel nicht: Neun Passagiere haben eine Bootsangabe und trotzdem `survived == 0`. Nach der Encyclopedia Titanica erreichten sie ein Boot und starben trotzdem, die meisten in den überfluteten Faltbooten A und B.) `home.dest` ist Freitext, den dieses Kursmodell nicht aufbereitet, und wird ebenfalls entfernt. `name`, `ticket` und `cabin` bleiben in der Tabelle, weil sie Passagiere identifizieren und später beim Prüfen helfen, sie werden aber nicht als Merkmal verwendet.

`pd.read_csv` liest eine Textdatei mit Komma als Trennzeichen und erkennt für jede Spalte selbst einen Datentyp. Mit den Standardeinstellungen führt das bei unseren Spalten zu: nur ganze Zahlen wird `int64`, Zahlen mit Nachkommastellen `float64`, und eine Spalte, in der auch nur ein Eintrag keine Zahl ist, wird Text. (Wahrheitswerte erkennt `read_csv` ebenfalls, in diesem Datensatz kommen sie nicht vor.) `encoding="utf-8"` legt fest, wie die Umlaute in der Datei zu lesen sind, wie bei `open` in den Ladezellen der früheren Lerneinheiten. Die nächste Zelle liest die Kursfassung ohne weitere Angaben ein und zeigt, was dabei herauskommt.

```
① raw = pd.read_csv("data/titanic_raw.csv", encoding="utf-8")
  print(raw.shape)
② print(raw.iloc[:3, :6])
  print()
③ raw.info()
```

① `pd.read_csv(pfad)` liest die Datei und liefert einen DataFrame; die erste Zeile der Datei wird zu den Spaltennamen
② `iloc[:3, :6]` wählt nach Position: die ersten drei Zeilen und die ersten sechs Spalten (Kapitel 4.3)
③ `info()` zeigt je Spalte die Anzahl der vorhandenen Werte und den Datentyp

```
(1321, 14)
   pclass  survived                             name     sex   age  sibsp
0       1         1    Allen, Miss. Elisabeth Walton  female  29.0      0
1       1         1   Allison, Master. Hudson Trevor    male  0.92      1
2       1         0     Allison, Miss. Helen Loraine  female   2.0      1

<class 'pandas.DataFrame'>
RangeIndex: 1321 entries, 0 to 1320
Data columns (total 14 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   pclass     1321 non-null   int64
 1   survived   1321 non-null   int64
 2   name       1321 non-null   str
 3   sex        1321 non-null   str
 4    age       1321 non-null   str
 5   sibsp      1321 non-null   int64
 6   parch      1321 non-null   int64
 7   ticket     1321 non-null   str
 8   fare       1321 non-null   str
 9   cabin      296 non-null    str
 10  embarked   1319 non-null   str
 11  boat       488 non-null    str
 12  body       122 non-null    float64
 13  home.dest  751 non-null    str
dtypes: float64(1), int64(4), str(9)
memory usage: 144.6 KB
```

> 💭 **Kurz überlegen**
>
> Welche Spalten müssen Zahlen sein, damit sich damit rechnen lässt? Welche davon sind es nach dem Einlesen nicht, und woran könnte das liegen?

```
  print(list(raw.columns))                              # zwei Namen mit Leerzeichen
  print(raw[" age"].head(6).tolist())                   # Zahlen, aber als Text (Anführungszeichen)
  print(raw.loc[raw[" age"] == "?", " age"].head(3).tolist())   # und so sehen die fehlenden Werte aus
  print("Einträge mit ?:", (raw[" age"] == "?").sum(), "in age,", (raw["fare "] == "?").sum(), "in fare")
```

```
['pclass', 'survived', 'name', 'sex', ' age', 'sibsp', 'parch', 'ticket', 'fare ', 'cabin',
'embarked', 'boat', 'body', 'home.dest']
['29.0', '0.92', '2.0', '30.0', '25.0', '48.0']
['?', '?', '?']
Einträge mit ?: 264 in age, 2 in fare
```

Zwei Angaben an `read_csv` beheben beides. `na_values=["?"]` sagt, dass `?` ein fehlender Wert ist. Die Spalte enthält dann nur noch Zahlen und Lücken und wird `float64`. Die Lücken selbst heißen `NaN` (not a number), ein besonderer Fließkommawert, den auch NumPy kennt. Ein `int64`-Array hat keine Darstellung für „fehlt", `NaN` ist ein Fließkommawert. Deshalb liest `read_csv` eine Zahlenspalte mit Lücken standardmäßig als `float64` ein, auch wenn alle vorhandenen Werte ganze Zahlen sind (die nullable Typen in der Übersicht unten wären die Alternative, wir verwenden sie nicht). Die Leerzeichen in den Spaltennamen entfernt `columns.str.strip()`, dieselbe Textmethode wie bei den Wörtern in Grundkurs Teil 2, hier auf die Namen angewendet.

```
① raw2 = pd.read_csv("data/titanic_raw.csv", encoding="utf-8", na_values=["?"])
② raw2.columns = raw2.columns.str.strip()
  print(raw2.dtypes)
  print()
③ print(raw2.isna().sum())
```

① `na_values=[...]` nennt zusätzliche Zeichenfolgen, die als fehlender Wert gelten; das leere Feld und einige Standardzeichenfolgen wie `NA` gelten mit den Standardeinstellungen ohnehin
② `columns` ist eine Folge wie eine Series; `.str.strip()` wirkt auf jeden Namen, die Zuweisung ersetzt die Namen
③ `isna()` liefert je Zelle True für fehlend; `sum()` zählt spaltenweise, wie eine Reduktion über Achse 0

```
pclass         int64
survived       int64
name             str
sex              str
age          float64
sibsp          int64
parch          int64
ticket           str
fare         float64
cabin            str
embarked         str
boat             str
body         float64
home.dest        str
dtype: object

pclass          0
survived        0
name            0
sex             0
age           264
sibsp           0
parch           0
ticket          0
fare            2
cabin        1025
embarked        2
boat          833
body         1199
home.dest     570
dtype: int64
```

Überblick: Fehlende Werte

Schreibweise	Bedeutung	Wo sie vorkommt
`NaN`	not a number, ein Fließkommawert; `np.nan` in NumPy	Zahlenspalten mit Lücken, beim Einlesen `float64`
`None`	das Python-Objekt für „kein Wert"	in Listen und Dicts; beim Anlegen eines DataFrames daraus wird es zur Lücke
`pd.NA`	pandas-eigener Platzhalter für Lücken	nur in den nullable Typen wie `Int64` und `boolean`; in diesem Kurs kommt er nicht vor
`isna()`, `notna()`	Maske: Zelle fehlt, Zelle ist vorhanden; erkennt alle drei Schreibweisen	auf Series und DataFrame
`dropna()`	Zeilen (oder Spalten) mit Lücken entfernen, als neues Objekt	vor dem Zeichnen, ohne die Gesamttabelle zu ersetzen

In diesem Notebook brauchen Sie nur `NaN`, `isna` und `dropna`. Die nullable Typen wie `Int64` (großes I) erlauben Lücken in einer Spalte, die trotzdem ganzzahlig bleibt; dafür markieren sie die Lücke mit `pd.NA` statt `NaN`, mit eigenen Regeln beim Vergleichen und Rechnen. Wir verwenden sie nicht: scikit-learn und PyTorch erwarten ohnehin Fließkomma-Arrays, `float64` mit `NaN` funktioniert überall gleich, und ein zweiter Platzhalter für Lücken wäre mehr zu lernen ohne Nutzen für diesen Kurs.

Vertiefung: Welche Parameter von read_csv braucht man?

NICHT PRÜFUNGSRELEVANT

`read_csv` hat über fünfzig Parameter. In der Praxis genügen meist vier: `sep` für ein anderes Trennzeichen (deutsche Exporte verwenden oft `;`), `encoding`, `na_values` und `usecols` für eine Auswahl der Spalten schon beim Einlesen.

Dazu kommen gelegentlich `dtype`, um einer Spalte den Typ vorzugeben (zum Beispiel eine Postleitzahl als Text, damit führende Nullen bleiben), `parse_dates` für Datumsspalten und `nrows`, um bei einer großen Datei zuerst nur die ersten Zeilen zu lesen. Alles Weitere schlagen Sie nach, wenn eine Datei es verlangt. Die Dokumentation von `read_csv` zeigt die vollständige Liste.

Steht in einer Zahlenspalte ein Eintrag, den `na_values` nicht kennt (etwa `unbekannt`), bleibt die Spalte Text. `pd.to_numeric(spalte, errors="coerce")` wandelt sie nachträglich um und macht aus jedem Eintrag, der keine Zahl ist, eine Lücke: `pd.to_numeric(pd.Series(["23", "unbekannt", "41"]), errors="coerce")` ergibt `23.0, NaN, 41.0`. Weil dabei aus Fehlern stillschweigend Lücken werden, zählen Sie danach die Lücken mit `isna().sum()` und vergleichen mit vorher. Ohne `errors="coerce"` bricht `to_numeric` beim ersten ungültigen Eintrag mit einem `ValueError` ab, was beim Nachforschen oft hilfreicher ist.

✏️ Aufgabe 4.2 #1

5 min

Schreiben Sie die Funktion `load_titanic(path)`. Sie liest die Datei unter `path` so ein, dass `age` und `fare` den Datentyp `float64` haben und alle Spaltennamen ohne Leerzeichen sind, und gibt den DataFrame zurück. Die Prüfzelle prüft Form, Spaltennamen und Datentypen. Spätere Zellen bauen auf `load_titanic` auf.

Hinweis: Zwei Angaben an `read_csv` und eine Zeile für die Spaltennamen. Das Ergebnis des Aufrufs `pd.read_csv(...)` ist der DataFrame, den Sie zurückgeben.

```python
def load_titanic(path):
    """Liest die Kursfassung des Titanic-Datensatzes als DataFrame ein."""
    # TODO: Ihre Lösung hier
    ...
```

```
    # Prüfzelle – einfach ausführen
    df = load_titanic("data/titanic_raw.csv")
    assert df is not None, "load_titanic gibt noch nichts zurück (return vergessen?)"
    assert df.shape == (1321, 14), f"die Kursfassung hat 1321 Zeilen und 14 Spalten, nicht {df.shape}"
    assert list(df.columns)[:5] == ["pclass", "survived", "name", "sex", "age"], "Spaltennamen ohne Leerzeichen
erwartet"
    assert df["age"].dtype == "float64" and df["fare"].dtype == "float64", "age und fare sollen float64 sein
(na_values)"
    assert df["age"].isna().sum() == 264, "264 fehlende Alterswerte erwartet (263 im Original plus eins in den
doppelten Zeilen)"
    import os, tempfile                                  # zweite Datei: die Funktion muss den übergebenen
Pfad lesen
    tiny_path = os.path.join(tempfile.gettempdir(), "titanic_tiny.csv")
    with open(tiny_path, "w", encoding="utf-8") as f:
        f.write("pclass, age\n1,?\n2,3.5\n")
    tiny = load_titanic(tiny_path)
    assert tiny.shape == (2, 2) and list(tiny.columns) == ["pclass", "age"], "load_titanic soll die Datei unter
path lesen und die Namen bereinigen"
    assert tiny["age"].dtype == "float64" and tiny["age"].isna().sum() == 1, "auch in der kleinen Datei soll ?
zur Lücke werden"
    print("Alle Prüfungen bestanden ✓")
```

Überblick: Einlesen und Überblick gewinnen

Die Ergebnisse gelten für die kleine Tabelle `small`; auf den Kursdaten stehen dieselben Aufrufe in den Zellen oben.

Absicht	Code	Ergebnis
CSV-Datei lesen	`pd.read_csv(pfad, encoding="utf-8")`	DataFrame
`?` als Lücke lesen	`pd.read_csv(pfad, na_values=["?"])`	Spalte wird `float64` statt Text
Leerzeichen aus Spaltennamen	`df.columns = df.columns.str.strip()`	ändert die Namen
erste Zeilen ansehen	`small.head(3)`	die ersten drei Zeilen
Form	`small.shape`	`(5, 3)`
Typ je Spalte	`small.dtypes`	`int64`, `str`, `float64`
Anzahl und Typ je Spalte	`small.info()`	Übersicht mit Lücken
Kennzahlen einer Zahlenspalte	`small["age"].describe()`	Anzahl 4, Mittelwert 25.44, …
Lücken je Spalte	`small.isna().sum()`	`age` 1, sonst 0

Falle: Steht in einer Zahlenspalte ein `?` oder ein Wort, liest `read_csv` die ganze Spalte als Text. `na_values` oder `pd.to_numeric(s, errors="coerce")` beheben das.

4.3 Auswählen und Filtern

Für die Bereinigung und die Auswertung der Daten müssen Sie oft Teile der Tabelle auswählen: eine Spalte, mehrere Spalten oder die Zeilen, die eine Bedingung erfüllen. Die Maske aus LE03 funktioniert hier genauso, nur mit Spaltennamen statt Achsen. Wie in LE03 üben wir zuerst an der kleinen Tabelle `small` mit bekannten Werten und gehen dann zu den Kursdaten. Die Grafik unten verwendet dieselben fünf Passagiere.

```
print(small)                          # die kleine Tabelle aus Kapitel 4.1
print()
① mask = small["pclass"] == 3
print(mask.tolist())
② print(small[mask])
```

① ein Vergleich mit einer Spalte liefert eine Series aus True und False: die Maske
② `df[maske]` behält die Zeilen, bei denen die Maske True ist; der Index der Zeilen bleibt erhalten

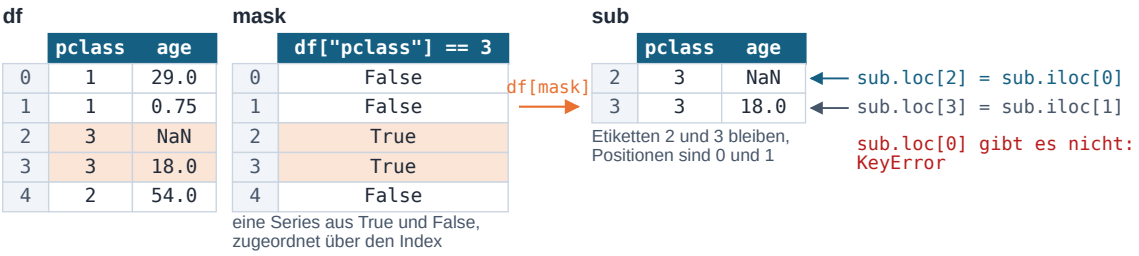
```
   pclass     sex    age
0       1  female  29.00
1       1    male   0.75
2       3    male    NaN
3       3  female  18.00
4       2    male  54.00

[False, False, True, True, False]
   pclass     sex   age
2       3    male   NaN
3       3  female  18.0
```

Die Maske sieht aus wie in LE03, und für Bedingungen gelten dieselben Regeln: `&` für und, `|` für oder, `~` für nicht, und jede Teilbedingung in Klammern, weil `&` stärker bindet als `==`. Ein Unterschied steckt im Ergebnis: Die Zeilen 2 und 3 behalten ihre Etiketten, also ihre Einträge im Index, hier die Nummern, die `pd.DataFrame` vergeben hat (es könnten auch Namen oder Datumsangaben sein). In NumPy wäre die erste Zeile des Ergebnisses einfach die Zeile 0. In pandas hat sie das Etikett 2 und die Position 0, und beides lässt sich ansprechen: `loc` wählt nach dem Etikett, `iloc` nach der Position.

> 💭 **Kurz überlegen**
>
> `sub = small[small["pclass"] == 3]` enthält die Zeilen mit den Etiketten 2 und 3. Was liefert `sub.iloc[0]`, was `sub.loc[0]`?

Eine Maske wählt Zeilen aus; loc fragt nach dem Etikett, iloc nach der Position



Ein NumPy-Array als Maske wird dagegen nach Position zugeordnet: Eintrag 0 gehört zur ersten Zeile, egal wie sie heißt.

```
   sub = small[small["pclass"] == 3]
① print("loc[2]:"); print(sub.loc[2])
   print("iloc[0]:"); print(sub.iloc[0])
② print("loc[mask, 'age']:", small.loc[small["sex"] == "female", "age"].tolist())
③ print(small.iloc[0:2, 0:2])
```

① `loc[etikett]` wählt die Zeile mit diesem Index-Etikett, `iloc[position]` die Zeile an dieser Position
② `loc[zeilen, spalten]` nimmt für beide Achsen Etiketten oder Masken, auch kombiniert
③ `iloc` nimmt Positionen wie ein Array: Zeilen 0 bis 1, Spalten 0 bis 1

```
loc[2]:
pclass       3
sex       male
age        NaN
Name: 2, dtype: object
iloc[0]:
pclass       3
sex       male
age        NaN
Name: 2, dtype: object
loc[mask, 'age']: [29.0, 18.0]
   pclass     sex
0       1  female
1       1    male
```

> 💭 **Kurz überlegen**
>
> `small.loc[1:3]` und `small.iloc[1:3]`: Wie viele Zeilen liefert jeder der beiden Ausdrücke?

```
   print("loc[1:3]: ", small.loc[1:3].index.tolist(), "<- Etiketten, Ende eingeschlossen")
   print("iloc[1:3]:", small.iloc[1:3].index.tolist(), "   <- Positionen, Ende ausgeschlossen wie in Python")
```

```
loc[1:3]:  [1, 2, 3] <- Etiketten, Ende eingeschlossen
iloc[1:3]: [1, 2]    <- Positionen, Ende ausgeschlossen wie in Python
```

Die Zuordnung über den Index gilt auch für die Maske selbst. Eine Maske als `Series` wird nicht nach Position angelegt, sondern nach Etikett: pandas sucht für jede Zeile der Tabelle den Eintrag mit demselben Etikett in der Maske. Ein NumPy-Array als Maske hat keine Etiketten und wird nach Position angelegt. Die nächste Zelle zeigt beides mit einer Maske, deren Index rückwärts läuft. pandas gibt dabei eine Warnung aus, dass es die Maske nach dem Index neu anordnet.

```
① mask_rev = pd.Series([True, False, False, False, False], index=[4, 3, 2, 1, 0])
   print("als Series:", small[mask_rev].index.tolist(), "<- True steht beim Etikett 4")
② print("als Array: ", small[mask_rev.to_numpy()].index.tolist(), "<- True steht an Position 0")
```

① dieselben Werte, aber die Etiketten laufen von 4 bis 0
② `to_numpy()` wirft die Etiketten weg, die Maske gilt dann nach Position

```
als Series: [4] <- True steht beim Etikett 4
als Array:  [0] <- True steht an Position 0
```

Sortieren und die größten Werte: `sort_values` ordnet nach einer Spalte, `nlargest` nimmt direkt die `n` größten. Beide geben ein neues Objekt zurück, die Tabelle bleibt unverändert. Das Beispiel läuft auf den Kursdaten `df` aus der Aufgabe oben.

```
① print(df.sort_values("fare", ascending=False).head(3)[["name", "pclass", "fare"]])
② print(df.nlargest(3, "age")[["name", "age"]])
```

① `sort_values("spalte", ascending=False)` ordnet absteigend; `head(3)` zeigt die ersten drei Zeilen des Ergebnisses
② `nlargest(n, "spalte")` ist die Abkürzung dafür

```
                                          name  pclass      fare
183                          Lesurer, Mr. Gustave J       1  512.3292
50   Cardeza, Mrs. James Warburton Martinez (Charlo...       1  512.3292
302                             Ward, Miss. Anna       1  512.3292
                                 name    age
727              Connors, Mr. Patrick  155.0
844     Hakkarainen, Mr. Pekka Pietari  134.0
14   Barkworth, Mr. Algernon Henry Wilson   80.0
```

Die beiden größten Alterswerte, 155 und 134 Jahre, sind unmöglich. Sie stehen in der Kursfassung absichtlich. Das Kapitel 4.4 setzt sie später auf fehlend.

✏️ Aufgabe 4.3 #1 Kurzübung

Was kommt heraus? Fünf Zeilen auf `small`, diesmal mit Masken, `loc` und `iloc`. Vorhersage hinter das `# ->`, ausführen, vergleichen, abwandeln.

```
sub = small[small["pclass"] == 3]
print((small["pclass"] == 1).tolist())                  # -> ?
print(small[small["sex"] == "female"].index.tolist())   # -> ?
print(sub.loc[3, "age"], sub.iloc[0]["age"])            # -> ?
print(small.loc[1:2, "sex"].tolist())                   # -> ?
print(small.iloc[1:2]["sex"].tolist())                  # -> ?
```

✏️ Aufgabe 4.3 #2 Kurzübung

Ersetzen Sie in der nächsten Zelle jedes `...` durch einen Ausdruck, keine Funktion. Die Teile a) bis e) arbeiten mit der kleinen Tabelle `small`, deren Ergebnis Sie vorher im Kopf bestimmen können, Teil f) mit den Kursdaten `df`.

- a) Die Spalte `age` von `small` als Series.
- b) Die Spalten `pclass` und `age` von `small` als DataFrame.
- c) Alle Zeilen von `small`, die zur dritten Klasse gehören.
- d) Das Alter aller Männer in `small`, ausgewählt mit `loc`, einer Maske und dem Spaltennamen.
- e) Aus dem Ergebnis von c) die zweite Zeile, nach Position.
- f) Wie viele Passagiere in `df` sind jünger als 10 Jahre und reisen in der dritten Klasse? Eine Zahl.

Die Prüfzelle meldet je Teil, was noch nicht passt.

Hinweis: a) und b) sind Spaltenauswahl mit einem Namen oder einer Liste von Namen, c) eine Maske in eckigen Klammern, d) `small.loc[maske, "age"]`, e) `iloc` auf dem Ergebnis von c), f) zwei Teilbedingungen in Klammern mit `&`, dann `.sum()`.

```
a = ...   # a) Spalte age von small als Series
b = ...   # b) Spalten pclass und age als DataFrame
c = ...   # c) alle Zeilen der dritten Klasse
d = ...   # d) Alter der Männer (loc, Maske, Spalte)
e = ...   # e) aus c: zweite Zeile nach Position
f = ...   # f) df: Kinder unter 10 in Klasse 3
```

```
# Prüfzelle – einfach ausführen
assert isinstance(a, pd.Series) and a.name == "age" and len(a) == 5, "a) die Spalte age von small ist eine Series mit fünf Werten"
assert isinstance(b, pd.DataFrame) and list(b.columns) == ["pclass", "age"], "b) zwei Spalten: eine Liste von Namen in den eckigen Klammern"
assert isinstance(c, pd.DataFrame) and list(c.index) == [2, 3], "c) die Zeilen mit pclass == 3; sie behalten die Etiketten 2 und 3"
assert isinstance(d, pd.Series) and list(d.index) == [1, 2, 4] and d.name == "age", "d) loc mit Maske und Spalte age: die Alterswerte der Männer, Etiketten 1, 2, 4"
assert isinstance(e, pd.Series) and e.name == 3 and e["age"] == 18.0, "e) c.iloc[1] ist die zweite Zeile von c, das ist die Zeile mit dem Etikett 3"
assert int(f) == 57, f"f) 57 Kinder unter 10 Jahren in der dritten Klasse (roh), nicht {f}"
print("Alle Prüfungen bestanden ✓")
```

💭 Kurz überlegen

Was passiert bei `df[df["pclass"] == 1 & df["age"] < 30]`, also ohne Klammern um die Teilbedingungen?

✏️ Aufgabe 4.3 #3 — 4 min

Schreiben Sie die Funktion `young_first_class_women(df)`. Sie gibt ein Tupel aus zwei Werten zurück: wie viele Frauen (`sex == "female"`) der ersten Klasse jünger als 30 Jahre sind, und ihr Durchschnittsalter. Die Prüfzelle rechnet auf den rohen Daten `df`. Das Ergebnis ist vorläufig: `df` enthält noch Duplikate und uneinheitliche Schreibweisen, im nächsten Kapitel rechnen Sie dieselbe Funktion auf den bereinigten Daten noch einmal.

Hinweis: Drei Teilbedingungen mit `&`, jede in Klammern. `mask.sum()` zählt die `True`-Einträge, `df.loc[mask, "age"].mean()` ist der Mittelwert der ausgewählten Alterswerte.

```
def young_first_class_women(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
result = young_first_class_women(df)
assert result is not None, "young_first_class_women gibt noch nichts zurück (return vergessen?)"
count, mean_age = result
assert count == 42, f"auf den rohen Daten sind es 42, nicht {count}"
assert abs(mean_age - 21.21) < 0.01, f"Durchschnittsalter rund 21.21, nicht {mean_age:.2f}"
print(f"vorläufig, auf den rohen Daten: {count} Frauen, Durchschnittsalter {mean_age:.2f}")
print("Alle Prüfungen bestanden ✓")
```

Überblick: Spalten auswählen und Zeilen filtern

Alle Ergebnisse an der Tabelle `small` mit den Etiketten 0 bis 4. Jede Zeile liefert ein neues Objekt, `small` bleibt unverändert.

Absicht	Code	Ergebnis
eine Spalte	`small["age"]`	Series, Form `(5,)`
eine Spalte als Tabelle	`small[["age"]]`	DataFrame, Form `(5, 1)`
mehrere Spalten	`small[["pclass", "age"]]`	DataFrame, Form `(5, 2)`
Maske bilden	`small["pclass"] == 3`	Series `False, False, True, True, False`
Zeilen auswählen	`small[small["pclass"] == 3]`	Zeilen mit den Etiketten 2 und 3

zwei Bedingungen	`erste = small["pclass"] == 1`
	`jung = small["age"] < 30`
	`small[erste & jung]`
		Zeilen 0 und 1

oder, nicht	`small[erste | jung]`, `small[~erste]`	Zeilen 0, 1, 3; Zeilen 2, 3, 4
Treffer zählen	`(small["sex"] == "male").sum()`	`3`
Maske mit Spalte	`small.loc[small["sex"] == "male", "age"]`	Series `0.75, NaN, 54.0`, Etiketten 1, 2, 4
nach Spalte ordnen	`small.sort_values("age", ascending=False)`	54.0 zuerst, `NaN` zuletzt
die größten n	`small.nlargest(2, "age")`	Zeilen 4 und 0

Falle: `small[small["pclass"] == 1 & small["age"] < 30]` ohne Klammern bricht mit `TypeError` ab, weil `&` stärker bindet als `==` und `<`.

Überblick: Etikett gegen Position

`loc` wählt nach dem Etikett im Index, `iloc` nach der Position. Nach einem Filter unterscheiden sich beide.

Absicht	Code	Ergebnis
Filterergebnis merken	`sub = small[small["pclass"] == 3]`	Etiketten 2 und 3
Zeile nach Etikett	`sub.loc[2]`	Zeile mit Etikett 2
Zeile nach Position	`sub.iloc[0]`	dieselbe Zeile
fehlendes Etikett	`sub.loc[0]`	`KeyError`
eine Zelle	`small.loc[3, "age"]`	`18.0`
Bereich nach Etikett	`small.loc[1:3]`	Etiketten 1, 2, 3: drei Zeilen, Ende eingeschlossen
Bereich nach Position	`small.iloc[1:3]`	Positionen 1, 2: zwei Zeilen, Ende ausgeschlossen
neue Etiketten 0 bis N−1	`sub.reset_index(drop=True)`	liefert Tabelle mit Etiketten 0, 1

Falle: Eine Maske als Series wird über die Etiketten zugeordnet, nicht über die Reihenfolge; ein NumPy-Array als Maske über die Position.

4.4 Bereinigen mit Vertrag

Beim Bereinigen legen Sie fest, welche Werte gültig sind, welche Schreibweisen vereinheitlicht und welche Spalten oder Zeilen entfernt werden. Damit diese Entscheidungen nachprüfbar sind, bekommt die bereinigte Tabelle einen Vertrag, so wie der Operator in LE03 einen hatte: eine Liste von Zusicherungen, welche Spalten die Tabelle hat, welchen Datentyp jede Spalte trägt, welche Werte darin zulässig sind und wo Lücken bleiben dürfen. Die Prüfzelle am Ende des Kapitels prüft genau diesen Vertrag.

Vertrag der bereinigten Tabelle

`clean_titanic(df)` nimmt den DataFrame aus `load_titanic` und gibt einen neuen DataFrame zurück. Die Eingabe bleibt unverändert. Der Rückgabewert erfüllt:

Spalte	Datentyp	Zulässige Werte	Lücken
`pclass`	`int64`	1, 2, 3	nein
`survived`	`int64`	0, 1	nein
`name`, `ticket`	`str`	beliebig	nein
`sex`	`str`	`female`, `male`	nein
`age`	`float64`	0 bis 100	ja
`sibsp`, `parch`	`int64`	ab 0	nein
`fare`	`float64`	ab 0	ja
`embarked`	`str`	`C`, `Q`, `S`	ja
`cabin`	`str`	beliebig	ja

Die Spalten `boat`, `body` und `home.dest` sind entfernt. Es gibt keine doppelten Zeilen. Der Index läuft von 0 bis N minus 1. Fehlende Werte bleiben fehlend, sie werden nicht aufgefüllt. `str` ist der Texttyp von pandas 3; unter älteren Versionen heißt derselbe Typ `object`. Geprüft werden die Werte, nicht der Name des Datentyps.

Warum die drei Spalten raus müssen, steht in der Spaltenübersicht: `boat` und `body` entstehen erst nach dem Unglück und gäbe es zum Vorhersagezeitpunkt nicht, und `home.dest` ist Freitext, den dieses Kursmodell nicht aufbereitet. Lücken füllen wir hier nicht. Der Wert zum Füllen, etwa der Mittelwert des Alters, wird aus den Daten berechnet. In der nächsten Lerneinheit darf dafür nur der Trainingsteil der Daten verwendet werden, deshalb passiert das Füllen erst dort, in der Pipeline. Hier zählen wir die Lücken und lassen sie stehen.

Die Werkzeuge, jeweils an der kleinen Tabelle oder an einem Ausschnitt der Kursdaten. `str.strip` und `str.lower` kennen Sie aus Grundkurs Teil 2 für einzelne Texte; über `.str` wirken sie hier auf alle Zellen einer Spalte in einem Aufruf, ohne Schleife.

```
① print(df["sex"].value_counts().to_dict())
  print(df["embarked"].value_counts(dropna=False).to_dict())
② print(df["sex"].str.strip().str.lower().value_counts().to_dict())
③ print(small["sex"].replace({"female": "w", "male": "m"}).tolist())
④ print("Alter über 100:", df.loc[df["age"] > 100, "age"].tolist(), "-> nach mask:", df["age"].mask(df["age"]
> 100).max())
⑤ print(df.drop(columns=["boat", "body", "home.dest"]).shape)
⑥ print("doppelte Zeilen:", df.duplicated().sum(), "-> drop_duplicates:", df.drop_duplicates().shape)
```

① `value_counts()` zeigt, welche Schreibweisen vorkommen; `dropna=False` zählt auch die Lücken
② `.str.strip().str.lower()` wendet beide Textmethoden auf jede Zelle an und liefert eine neue Series
③ `replace({alt: neu})` ersetzt einzelne Werte nach einer Zuordnung; hier nur gezeigt, für die Schreibweisen reicht strip und lower
④ `mask(bedingung)` ersetzt die Einträge, bei denen die Bedingung True ist, durch NaN (das Gegenstück `where` behält sie)
⑤ `drop(columns=[...])` liefert eine neue Tabelle ohne diese Spalten
⑥ `duplicated()` markiert jede Zeile, die genauso schon einmal vorkam; `drop_duplicates()` entfernt sie

```
{'male': 826, 'female': 455, 'Male': 8, ' male': 8, 'male ': 7, ' female': 5, 'female ': 5, 'Female':
4, 'MALE': 2, 'FEMALE': 1}
{'S': 898, 'C': 269, 'Q': 122, 's': 10, 'S ': 7, ' s': 6, ' c': 3, 'C ': 2, nan: 2, 'Q ': 2}
{'male': 851, 'female': 470}
['w', 'm', 'm', 'w', 'm']
Alter über 100: [155.0, 134.0] -> nach mask: 80.0
(1321, 11)
doppelte Zeilen: 11 -> drop_duplicates: (1310, 14)
```

> 💭 **Kurz überlegen**
>
> Die Kursfassung hat zwölf doppelte Zeilen, `duplicated` findet aber nur elf. Erst die Duplikate entfernen und dann die Schreibweisen vereinheitlichen, oder umgekehrt?

Zuweisen. Beim Bereinigen schreiben Sie in die Tabelle, dafür brauchen Sie zwei Formen. `df["age"] = neue_werte` ersetzt die ganze Spalte oder legt eine neue an, auch mit einem anderen Datentyp. `df.loc[mask, "age"] = wert` ändert nur die ausgewählten Zellen. Einen Wert, der nicht in den Datentyp der Spalte passt (etwa `1.5` in eine `int64`-Spalte), weist pandas 3 mit einem `TypeError` ab. Die Ausnahme ist `NaN`: Ein fehlender Wert in einer `int64`-Spalte macht die ganze Spalte `float64`, weil Ganzzahlen keine Lücke kennen (siehe Überblick Fehlende Werte).

Eine dritte Form sieht aus, als würde sie funktionieren, tut es aber nicht: `df[mask]["age"] = wert`. Das ist eine verkettete Zuweisung. `df[mask]` liefert zuerst ein Zwischenergebnis, eine neue Tabelle mit den ausgewählten Zeilen, und die Zuweisung schreibt in dieses Zwischenergebnis, nicht in `df`. In pandas 3 bleibt `df` unverändert, und pandas gibt eine Warnung aus. Der Grund ist die Regel Copy-on-Write, nach der pandas 3 arbeitet: Ein Objekt, das aus einem anderen abgeleitet ist, etwa ein Filterergebnis oder eine herausgegriffene Spalte, verhält sich beim Schreiben wie eine eigene Kopie. Wo sich zwei Objekte dabei Speicher teilen (bei einer herausgegriffenen Spalte zum Beispiel), kopiert pandas vor dem ersten Schreiben, daher der Name; ein Filterergebnis ist oft schon beim Filtern eine Kopie. Für Sie heißt das: Ein Filterergebnis zu verändern verändert nie die Tabelle, aus der es stammt. Wer in `df` schreiben will, schreibt mit `df.loc[mask, "age"] = wert`.

```
   t = small.copy()
① t["age"] = t["age"].astype("float32")
   print("Spalte ersetzt:", t["age"].dtype)
② t.loc[t["pclass"] == 3, "age"] = 0.0
   print("Zellen geändert:", t["age"].tolist(), t["age"].dtype)
③ t.loc[t["pclass"] == 2, "pclass"] = np.nan
   print("NaN in int64-Spalte:", t["pclass"].tolist(), t["pclass"].dtype)
④ sub = t[t["pclass"] == 1]
   sub["age"] = 99.0
   print("t nach dem Schreiben in sub:", t["age"].tolist())
⑤ with warnings.catch_warnings(record=True) as w:
       warnings.simplefilter("always")
       t[t["pclass"] == 1]["age"] = 99.0
   print("verkettet:", t["age"].tolist(), "| Warnung:", type(w[0].message).__name__ if w else "keine")
```

① ganze Spalte ersetzen: der Datentyp darf sich ändern; `astype(typ)` liefert die Spalte mit diesem Datentyp (hier float32 statt float64)
② ausgewählte Zellen ändern: `loc[maske, spalte] = wert`, der Datentyp der Spalte bleibt
③ die Ausnahme: ein fehlender Wert in einer int64-Spalte macht die Spalte float64, weil Ganzzahlen keine Lücke kennen
④ ein Filterergebnis ist unter Copy-on-Write unabhängig: Schreiben darauf lässt t unverändert
⑤ die verkettete Zuweisung ändert nichts; `warnings.catch_warnings` fängt die Warnung ab, damit sie hier als Text erscheint

```
Spalte ersetzt: float32
Zellen geändert: [29.0, 0.75, 0.0, 0.0, 54.0] float32
NaN in int64-Spalte: [1.0, 1.0, 3.0, 3.0, nan] float64
t nach dem Schreiben in sub: [29.0, 0.75, 0.0, 0.0, 54.0]
verkettet: [29.0, 0.75, 0.0, 0.0, 54.0] | Warnung: ChainedAssignmentError
```

Die Zuordnung über den Index gilt auch beim Zuweisen einer Spalte. `df["col"] = series` legt die Werte nicht der Reihe nach ab, sondern sucht für jede Zeile von `df` den Eintrag mit demselben Etikett in der Series. Gleiche Länge heißt also nicht gleiche Reihenfolge. Ein NumPy-Array oder eine Liste hat keine Etiketten und wird nach Position zugewiesen.

💭 Kurz überlegen

`t["rank"] = pd.Series([1, 2, 3, 4, 5], index=[4, 3, 2, 1, 0])` auf einer Kopie von `small`: Welchen Wert bekommt die Zeile mit dem Etikett 0? Und bei `t["rank"] = np.array([1, 2, 3, 4, 5])`?

```
   t = small.copy()
① t["rank"] = pd.Series([1, 2, 3, 4, 5], index=[4, 3, 2, 1, 0])
   print("als Series:", t["rank"].tolist())
② t["rank"] = np.array([1, 2, 3, 4, 5])
   print("als Array: ", t["rank"].tolist())
③ t["rank"] = pd.Series([1, 2, 3], index=[0, 1, 2])
   print("zu kurz:   ", t["rank"].tolist())
```

① eine Series mit rückwärts laufenden Etiketten: zugewiesen wird über das Etikett, nicht über die Position
② ein Array hat keine Etiketten und füllt die Spalte der Reihe nach
③ fehlt ein Etikett in der Series, bleibt die Zelle leer

```
als Series: [5, 4, 3, 2, 1]
als Array:  [1, 2, 3, 4, 5]
zu kurz:    [1.0, 2.0, 3.0, nan, nan]
```

✏️ Aufgabe 4.4 #1 Kurzübung

Was kommt heraus? Die nächste Zelle arbeitet auf einer Kopie `t` von `small`. Vorhersage hinter das `# ->`, ausführen, vergleichen. Die letzte Zeile ist die wichtigste.

```
t = small.copy()
print(t["sex"].str.upper().tolist()[:2])      # -> ?
print(t["age"].mask(t["age"] < 1).tolist())   # -> ?
t.loc[t["pclass"] == 1, "age"] = 0.0
print(t["age"].tolist())                      # -> ?
print(t.drop(columns=["age"]).shape)          # -> ?
print(small["age"].tolist())                  # -> ?
```

✏️ Aufgabe 4.4 #2

Kurzübung

Die nächste Zelle legt drei kleine Objekte an: eine Textspalte `s` mit uneinheitlichen Schreibweisen, eine Altersspalte `ages` mit einem unmöglichen Wert und einer Lücke, und eine Tabelle `three` mit einer doppelten Zeile. Ersetzen Sie jedes `...` durch einen Ausdruck, der ein Werkzeug der Bereinigung einzeln anwendet:

- a) `s` mit vereinheitlichten Schreibweisen: ohne Leerzeichen am Rand und in Kleinbuchstaben.
- b) `ages` mit dem unmöglichen Wert über 100 auf fehlend gesetzt; die vorhandene Lücke bleibt.
- c) `small` ohne die Spalte `age`; `small` selbst bleibt unverändert.
- d) Die Anzahl der doppelten Zeilen in `three`, eine Zahl.

Hinweis: a) `.str.strip().str.lower()`, b) `.mask(bedingung)`, c) `.drop(columns=[...])`, d) `.duplicated().sum()`.

```
s = pd.Series([" Female", "male", "MALE ", None])
ages = pd.Series([22.0, 140.0, np.nan, 5.0])
three = pd.DataFrame({"x": [1, 1, 2], "y": ["a", "a", "b"]})
a = ...   # a) Schreibweisen in s vereinheitlichen
b = ...   # b) unmögliche Werte in ages -> fehlend
c = ...   # c) small ohne age, small unverändert
d = ...   # d) doppelte Zeilen in three zählen
```

```
# Prüfzelle — einfach ausführen
assert isinstance(a, pd.Series) and a.tolist()[:3] == ["female", "male", "male"] and pd.isna(a[3]), "a) female, male, male und die Lücke bleibt eine Lücke"
assert isinstance(b, pd.Series) and b.isna().sum() == 2 and b.max() == 22.0, "b) 140 wird fehlend, die Lücke bleibt, 22 und 5 bleiben"
assert isinstance(c, pd.DataFrame) and list(c.columns) == ["pclass", "sex"] and "age" in small.columns, "c) nur age entfernt, und small hat die Spalte noch"
assert int(d) == 1, f"d) eine doppelte Zeile, nicht {d}"
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 4.4 #3

7 min

Schreiben Sie `clean_titanic(df)` nach dem Vertrag. Die fünf Schritte stehen als Gerüst in der Zelle, jede Zeile ein Werkzeug aus der Kurzübung. Die Prüfzelle prüft jeden Punkt des Vertrags. Sie prüft außerdem, dass `df` unverändert bleibt. Spätere Zellen bauen auf `clean_titanic` auf.

Wer früher fertig ist: `ticket` enthält bei vielen Fahrkarten ein Präfix vor der Nummer, etwa `"PC 17599"`. Schreiben Sie in einer neuen Zelle eine Spalte `ticket_no`, die nur den letzten durch Leerzeichen getrennten Teil enthält (`.str.split()` liefert je Zelle eine Liste, `.str[-1]` ihr letztes Element), und zählen Sie, wie viele Fahrkarten ein Präfix haben.

Hinweis: Die Werkzeuge stehen in der Kurzübung und in der Zelle über der Denkfrage; `embarked` wird mit `str.upper()` vereinheitlicht. `drop(columns=...)` liefert schon eine neue Tabelle, in die Sie dann schreiben dürfen, ohne `df` zu verändern. Die Reihenfolge der Schritte 2 bis 5 ist Teil des Vertrags, siehe die Denkfrage zu den Duplikaten.

```python
def clean_titanic(df):
    """Bereinigt die Kursfassung nach dem Vertrag und gibt einen neuen DataFrame zurück."""
    out = ...               # 1. ohne boat, body, home.dest
    out["sex"] = ...        # 2. sex vereinheitlichen
    out["embarked"] = ...   # 3. embarked, Großbuchstaben
    out["age"] = ...        # 4. unmögliche Alter -> fehlend
    out = ...               # 5. Duplikate raus, neuer Index
    return out
```

```python
# Prüfzelle – einfach ausführen
df = load_titanic("data/titanic_raw.csv")            # frischer Stand, falls ein früherer Versuch df verändert hat
before = df.copy()
clean = clean_titanic(df)
assert clean is not None, "clean_titanic gibt noch nichts zurück (return vergessen?)"
assert df.equals(before), "die Eingabe df wurde verändert; clean_titanic soll eine neue Tabelle liefern"
assert clean.shape == (1309, 11), f"1309 Zeilen und 11 Spalten erwartet, nicht {clean.shape}"
for col in ["boat", "body", "home.dest"]:
    assert col not in clean.columns, f"{col} soll entfernt sein"
def is_text(s):                                      ①
    return all(isinstance(v, str) for v in s.dropna())
assert is_text(clean["sex"]) and set(clean["sex"]) == {"female", "male"}, "sex: nur female und male"
assert set(clean["embarked"].dropna()) == {"C", "Q", "S"}, "embarked: nur C, Q, S oder fehlend"
assert clean["embarked"].isna().sum() == 2, "die zwei fehlenden Häfen bleiben fehlend"
assert clean["age"].dtype == "float64" and clean["age"].max() <= 100 and clean["age"].min() >= 0, "age zwischen 0 und 100 oder fehlend"
assert clean["age"].isna().sum() == 265, "265 fehlende Alterswerte: 263 aus dem Original plus die zwei unmöglichen"
assert clean["fare"].dtype == "float64" and clean["fare"].min() >= 0, "fare ab 0 oder fehlend"
assert clean["fare"].isna().sum() == 1, "der eine fehlende Fahrpreis bleibt fehlend (nicht auffüllen)"
assert clean["cabin"].isna().sum() == 1014, "die 1014 fehlenden Kabinen bleiben fehlend (nicht auffüllen)"
for col in ["name", "ticket", "cabin"]:
    assert is_text(clean[col]), f"{col} soll Text bleiben"
assert clean["name"].isna().sum() == 0 and clean["ticket"].isna().sum() == 0, "name und ticket ohne Lücken"
assert set(clean["pclass"]) == {1, 2, 3} and set(clean["survived"]) == {0, 1}, "pclass 1 bis 3, survived 0 oder 1"
for col in ["pclass", "survived", "sibsp", "parch"]:
    assert clean[col].dtype == "int64", f"{col} soll int64 bleiben"
assert clean["sibsp"].min() >= 0 and clean["parch"].min() >= 0, "sibsp und parch ab 0"
assert set(zip(clean["name"], clean["ticket"])) == set(zip(df["name"], df["ticket"])), "es fehlen Passagiere oder es kamen welche dazu"                                      ②
assert clean.duplicated().sum() == 0, "keine doppelten Zeilen"
assert list(clean.index) == list(range(len(clean))), "Index von 0 bis N-1 (reset_index)"
orig = df.drop_duplicates(subset=["name", "ticket"]).set_index(["name", "ticket"])                                                                                          ③
got = clean.set_index(["name", "ticket"])
orig = orig.loc[got.index]
for col in ["pclass", "survived", "sibsp", "parch"]:
    assert (got[col].to_numpy() == orig[col].to_numpy()).all(), f"{col}: Werte wurden verändert"
assert np.allclose(got["fare"].to_numpy(), orig["fare"].to_numpy(), equal_nan=True), "fare: Werte wurden verändert (nur fehlende Werte bleiben fehlend, sonst nichts anfassen)"
ok_age = orig["age"].to_numpy() <= 100
assert np.allclose(got["age"].to_numpy()[ok_age], orig["age"].to_numpy()[ok_age], equal_nan=True), "age: nur die zwei unmöglichen Werte werden fehlend, alle anderen bleiben"
print("Alle Prüfungen bestanden ✓")
```

① Text wird an den Werten geprüft, nicht am Namen des Datentyps (str unter pandas 3, object unter älteren Versionen)
② gültige Werte bleiben erhalten: jede Kombination aus Name und Fahrkarte der Eingabe kommt in der Ausgabe vor
③ gültige Werte bleiben unverändert: je Passagier (Name und Fahrkarte) müssen Klasse, survived, sibsp, parch, Fahrpreis und plausibles Alter denen der Eingabe gleichen

```
# Lösung des Zusatzes (ticket_no) in einer eigenen Zelle
# TODO: Ihre Lösung hier
...
```

✏️ Aufgabe 4.4 #4 2 min

Rufen Sie `young_first_class_women` aus dem letzten Kapitel noch einmal auf, diesmal mit der bereinigten Tabelle `clean`, und vergleichen Sie mit dem vorläufigen Ergebnis. Die Prüfzelle zeigt beide Werte nebeneinander.

```
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen
count_raw, mean_raw = young_first_class_women(df)
count_clean, mean_clean = young_first_class_women(clean)
assert count_clean == 43 and abs(mean_clean - 21.14) < 0.01, "bereinigt: 43 Frauen, Durchschnittsalter rund 21.14"
print(f"roh: {count_raw} Frauen, {mean_raw:.2f} Jahre   |   bereinigt: {count_clean} Frauen, {mean_clean:.2f} Jahre")
print("Alle Prüfungen bestanden ✓")
```

Der Filtercode war auch auf den rohen Daten richtig, das Ergebnis trotzdem falsch: Eine Zeile mit `" female"` (mit Leerzeichen davor) wurde nicht mitgezählt. Erst auf den bereinigten Daten liefert dieselbe Funktion das richtige Ergebnis.

Überblick: Werte und Spalten ändern

Für Änderungen verwenden wir diese beiden Zuweisungsformen; die Methoden darunter liefern neue Objekte. Die Zeilen arbeiten auf einer Kopie `beispiel`, damit `small` für alle Beispiele gleich bleibt.

Absicht	Code	Ergebnis
Kopie zum Ausprobieren	`beispiel = small.copy()`	eigene Tabelle, `small` bleibt
ganze Spalte ersetzen oder anlegen	`beispiel["age_rounded"] = beispiel["age"].round()`	ändert `beispiel`, neue Spalte (weiter `float64`)
ausgewählte Zellen	`beispiel.loc[beispiel["pclass"] == 3, "age"] = 0.0`	ändert zwei Zellen
Datentyp einer Spalte	`beispiel["age"] = beispiel["age"].astype("float32")`	ändert `beispiel`
Leerzeichen weg, klein	`beispiel["sex"].str.strip().str.lower()`	liefert Series
Werte nach Zuordnung	`beispiel["sex"].replace({"male": "m"})`	liefert Series
Werte unter Bedingung auf fehlend	`beispiel["age"].mask(beispiel["age"] < 1)`	liefert Series, `0.75` wird `NaN`
Spalten entfernen	`beispiel.drop(columns=["age_rounded"])`	liefert Tabelle
doppelte Zeilen entfernen	`beispiel.drop_duplicates()`	liefert Tabelle, zuletzt anwenden
neue Etiketten	`beispiel.reset_index(drop=True)`	liefert Tabelle mit Etiketten 0 bis N−1
neues Objekt übernehmen	`beispiel = beispiel.drop(columns=["age_rounded"])`	ersetzt `beispiel`

Fehlerfall: `beispiel[beispiel["pclass"] == 3]["age"] = 0.0` ändert nichts und gibt eine Warnung aus. Der Filter liefert ein Zwischenergebnis, die Zuweisung schreibt dorthin (Copy-on-Write: ein Filterergebnis ist von seiner Quelle unabhängig). Richtig ist die `loc`-Form.

Falle: `beispiel["col"] = series` ordnet über die Etiketten zu, nicht über die Reihenfolge; ein Array oder eine Liste wird nach Position zugewiesen. Ein `NaN` in einer `int64`-Spalte macht die Spalte `float64`.

4.5 Gruppieren und Zusammenfassen

Oft interessiert nicht der Mittelwert einer ganzen Spalte, sondern der Mittelwert je Gruppe: die Überlebensrate je Klasse, das Durchschnittsalter je Geschlecht, die Anzahl der Passagiere je Hafen. Das ist dieselbe Reduktion wie in LE03, nur nicht entlang einer Achse, sondern je Gruppe. In pandas sind das zwei Schritte: `groupby` legt die Gruppen fest, und erst `mean`, `sum` oder `size` fasst jede Gruppe zusammen. Die erste Zelle rechnet an sechs Zeilen mit bekannten Werten, denselben wie in der Grafik. Dass dort `mean` auf der Spalte `survived` steht, hat einen Grund: Der Mittelwert einer 0/1-Spalte ist der Anteil der Einsen, also die Überlebensrate. Mit `mean` bekommen wir die Rate, ohne zu zählen und zu teilen.

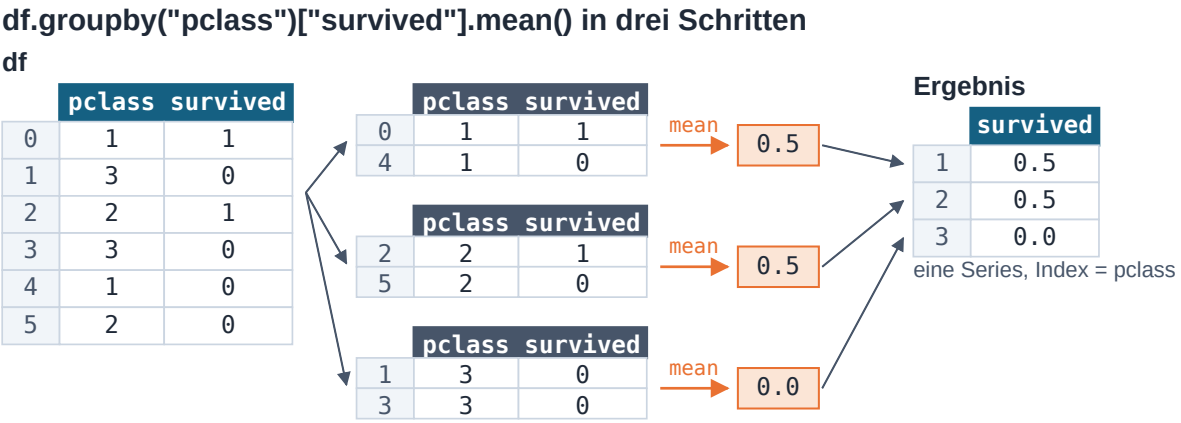


groupby ordnet jede Zeile dem Fach ihres Gruppenschlüssels zu; das ist kein Sortieren, die Zeilen werden nur verteilt. Erst die Zusammenfassung danach, hier das Zählen je Fach, liefert eine Zahl je Gruppe. Mit mean oder sum statt Zählen entsteht je Fach eine andere Kennzahl.

```
six = pd.DataFrame({"pclass": [1, 3, 2, 3, 1, 2], "survived": [1, 0, 1, 0, 0, 0]})
① print(six.groupby("pclass")["survived"].mean())
② s = pd.Series([1.0, 0.0, np.nan])
print("mean:", s.mean(), "| count:", s.count(), "| size:", s.size)
```

① `groupby("pclass")` legt die Gruppen fest, `["survived"]` wählt die Spalte, `mean()` fasst jede Gruppe zusammen
② `mean` lässt fehlende Werte aus (Parameter `skipna`, Voreinstellung True): eine Lücke zählt weder oben noch unten

```
pclass
1    0.5
2    0.5
3    0.0
Name: survived, dtype: float64
mean: 0.5 | count: 2 | size: 3
```



Dieselbe Reduktion wie in NumPy, nur je Gruppe statt entlang einer Achse. Fehlende Werte lässt mean aus.

Das Ergebnis ist eine `Series`, deren Index die Gruppenschlüssel sind. Auf den Kursdaten dasselbe, dazu zwei Dinge, die bei fehlenden Werten wichtig sind: `mean` lässt Lücken aus, wie die kleine Series gezeigt hat, es teilt also durch die Anzahl der vorhandenen Werte. `size` zählt alle Zeilen einer Gruppe, `count` nur die vorhandenen Werte der gewählten Spalte.

```
print(clean.groupby("pclass")["survived"].mean().round(3))
print()
① print(pd.DataFrame({"size": clean.groupby("pclass")["age"].size(),
                      "count": clean.groupby("pclass")["age"].count(),
                      "mean": clean.groupby("pclass")["age"].mean().round(1)}))
```

① `size()` zählt Zeilen je Gruppe, `count()` zählt nur Werte, die nicht fehlen

```
pclass
1    0.619
2    0.430
3    0.255
Name: survived, dtype: float64

        size  count  mean
pclass
1        323    284  39.2
2        277    261  29.5
3        709    499  24.7
```

Lücken gibt es auch im Gruppenschlüssel selbst. Zeilen, deren Schlüssel fehlt, lässt `groupby` standardmäßig weg. Sie landen in keiner Gruppe. `dropna=False` legt für sie eine eigene Gruppe an.

> 💭 **Kurz überlegen**
>
> `clean.groupby("embarked").size().sum()` ergibt 1307, die Tabelle hat 1309 Zeilen. Wo sind die zwei Passagiere geblieben?

```
① by_port = clean.groupby("embarked").size()
  print(by_port.to_dict(), "Summe:", by_port.sum())
② by_port_all = clean.groupby("embarked", dropna=False).size()
  print(by_port_all.to_dict(), "Summe:", by_port_all.sum())
```

① Zeilen mit fehlendem Gruppenschlüssel fallen bei `groupby` standardmäßig weg
② `dropna=False` legt für die fehlenden Schlüssel eine eigene Gruppe an

```
{'C': 270, 'Q': 123, 'S': 914} Summe: 1307
{'C': 270, 'Q': 123, 'S': 914, nan: 2} Summe: 1309
```

Mit zwei Gruppenschlüsseln entsteht eine Series mit einem zweistufigen Index (ein MultiIndex), ein Eintrag je Kombination. `unstack` nimmt die innere Stufe und macht daraus Spalten, das Ergebnis ist eine Tabelle, wie man sie lesen möchte.

> 💭 **Kurz überlegen**
>
> Welche Form hat `clean.groupby(["sex", "pclass"])["survived"].mean()`?

```
① by_both = clean.groupby(["sex", "pclass"])["survived"].mean()
  print(by_both.round(3))
  print("Form:", by_both.shape)
② print(by_both.unstack().round(3))
```

① eine Liste von Spalten gruppiert nach allen Kombinationen; das Ergebnis hat einen zweistufigen Index
② `unstack()` macht aus der inneren Indexstufe (pclass) Spalten

```
sex     pclass
female  1         0.965
        2         0.887
        3         0.491
male    1         0.341
        2         0.146
        3         0.152
Name: survived, dtype: float64
Form: (6,)
pclass      1      2      3
sex
female  0.965  0.887  0.491
male    0.341  0.146  0.152
```

Eine Schleife über die Zeilen (`for i, row in df.iterrows(): …`) würde dasselbe Ergebnis liefern, aber sie hat dasselbe Problem wie die Python-Schleife in LE03: Jeder Zugriff erzeugt ein Python-Objekt, und der Interpreter arbeitet jede Runde einzeln ab. `groupby` rechnet dagegen auf ganzen Arrays. Der Unterschied wächst mit der Zeilenzahl. In LE03 haben Sie ihn an der Schleife über die Zeilen eines Arrays gemessen.

Vertiefung: agg, pivot_table und crosstab

`agg` fasst mit mehreren Kennzahlen auf einmal zusammen, `pivot_table` ist `groupby` mit `unstack` in einem Aufruf, und `crosstab` zählt Häufigkeiten zweier Spalten gegeneinander: `pd.crosstab(small["sex"], small["pclass"])` ergibt eine Tabelle mit `female` und `male` in den Zeilen, den Klassen 1, 2, 3 in den Spalten und den Zählwerten 1, 0, 1 und 1, 1, 1.

`clean.groupby("pclass")["age"].agg(["mean", "median", "count"])` liefert drei Spalten. `clean.pivot_table(values="survived", index="sex", columns="pclass", aggfunc="mean")` ist die Tabelle aus der Zelle oben. `pd.crosstab(clean["sex"], clean["pclass"])` zählt, wie viele Frauen und Männer je Klasse. Es liefert Häufigkeiten (Anzahlen), keine Raten (Anteile wie die Überlebensrate). Für Raten braucht `crosstab` zusätzlich `values=clean["survived"], aggfunc="mean"`. Im Termin genügen `groupby`, `mean`, `size` und `unstack`; `crosstab` und `agg` kommen in der Projektaufgabe vor.

✏️ Aufgabe 4.5 #1 Kurzübung

Was kommt heraus? Vier `groupby`-Zeilen auf `small`. Vorhersage hinter das `# ->`, ausführen, vergleichen. Achten Sie auf die Lücke im Alter.

```
print(small.groupby("pclass").size().to_dict())                    # -> ?
print(small.groupby("sex")["age"].mean().round(2).to_dict())       # -> ?
print(small.groupby("pclass")["age"].count().to_dict())            # -> ?
print(small.groupby(["pclass", "sex"]).size().unstack(fill_value=0))   # -> ?
```

✏️ Aufgabe 4.5 #2 Kurzübung

Eine Gruppe, eine Kennzahl. Ersetzen Sie jedes `...` durch einen Ausdruck auf der bereinigten Tabelle `clean`, jeweils ein `groupby` mit einer Zusammenfassung:

- a) Wie viele Passagiere reisen in jeder Klasse? Eine Series mit der Klasse im Index.
- b) Wie alt sind Frauen und Männer im Durchschnitt? Eine Series mit dem Geschlecht im Index.
- c) Wie hoch ist die Überlebensrate je Einstiegshafen? Eine Series mit dem Hafen im Index.

Hinweis: a) `size()` direkt auf dem `groupby`, b) und c) erst die Spalte wählen (`["age"]`, `["survived"]`), dann `mean()`; der Mittelwert einer 0/1-Spalte ist der Anteil der Einsen.

```
a = ...   # a) Passagiere je Klasse
b = ...   # b) Durchschnittsalter je Geschlecht
c = ...   # c) Überlebensrate je Hafen
```

```
# Prüfzelle – einfach ausführen
assert isinstance(a, pd.Series) and a.to_dict() == {1: 323, 2: 277, 3: 709}, "a) 323, 277 und 709 Passagiere, Index = Klasse"
assert isinstance(b, pd.Series) and b.round(2).to_dict() == {"female": 28.69, "male": 30.53}, f"b) Frauen im Mittel 28.69 Jahre, Männer 30.53, nicht {b.round(2).to_dict()}"
assert isinstance(c, pd.Series) and c.round(3).to_dict() == {"C": 0.556, "Q": 0.358, "S": 0.333}, f"c) Cherbourg 55.6 %, Queenstown 35.8 %, Southampton 33.3 %, nicht {c.round(3).to_dict()}"
print("je Klasse:", a.to_dict(), "| Alter:", b.round(1).to_dict(), "| Rate:", c.round(3).to_dict())
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 4.5 #3 5 min

Schreiben Sie `survival_table(df)`. Sie gibt die Überlebensrate nach Geschlecht und Klasse als Tabelle zurück, mit den Geschlechtern als Zeilen und den Klassen 1, 2, 3 als Spalten. Die Prüfzelle prüft Form und Werte.

Hinweis: `groupby` mit zwei Schlüsseln, dann `unstack` auf das Ergebnis.

```
def survival_table(df):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
table = survival_table(clean)
assert table is not None, "survival_table gibt noch nichts zurück (return vergessen?)"
assert table.shape == (2, 3), f"zwei Zeilen (Geschlecht) und drei Spalten (Klasse) erwartet, nicht {table.shape}"
assert list(table.index) == ["female", "male"] and list(table.columns) == [1, 2, 3], "Zeilen female/male, Spalten 1, 2, 3"
for sx in table.index:                                  # alle sechs Werte gegen eine Masken-Referenz
    for pc in table.columns:
        mask = (clean["sex"] == sx) & (clean["pclass"] == pc)
        assert abs(table.loc[sx, pc] - clean.loc[mask, "survived"].mean()) < 1e-9, f"{sx}, Klasse {pc}: Überlebensrate stimmt nicht"
assert abs(table.loc["female", 1] - 0.965) < 0.001, "Frauen in der ersten Klasse: rund 96,5 %"
print(table.round(3))
print("Alle Prüfungen bestanden ✓")
```

Überblick: Gruppieren und Zusammenfassen

`groupby` legt die Gruppen fest, erst `mean`, `sum`, `size` oder `count` fasst jede Gruppe zu einer Zahl zusammen. Bei einer ausgewählten Spalte entsteht eine Series mit den Gruppenschlüsseln als Index; ohne Spaltenauswahl ein DataFrame. Ergebnisse an `small`.

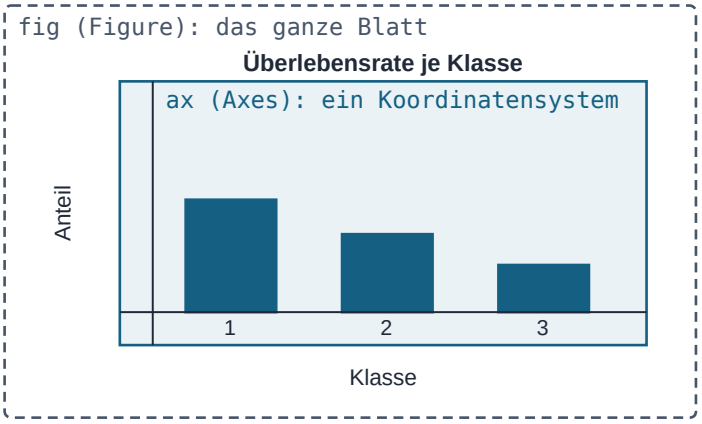
Absicht	Code	Ergebnis
Mittelwert je Gruppe	`small.groupby("sex")["age"].mean()`	`female 23.5, male 27.375`, Lücke ausgelassen
Zeilen je Gruppe	`small.groupby("pclass").size()`	`1: 2, 2: 1, 3: 2`
vorhandene Werte je Gruppe	`small.groupby("pclass")["age"].count()`	`1: 2, 2: 1, 3: 1`
zwei Schlüssel	`je = small.groupby(["pclass", "sex"]).size()`	Series mit zweistufigem Index
als Tabelle	`je.unstack(fill_value=0)`	Zeilen `pclass`, Spalten `sex`
Häufigkeiten gegeneinander	`pd.crosstab(small["sex"], small["pclass"])`	Tabelle 2 × 3
Häufigkeit je Wert	`small["sex"].value_counts()`	`male 3, female 2`, absteigend
Kennzahlen einer Zahlenspalte	`small["age"].describe()`	Anzahl, Mittelwert, Streuung, Quantile

Falle: Zeilen mit fehlendem Gruppenschlüssel lässt `groupby` weg; `dropna=False` legt für sie eine eigene Gruppe an. Der Mittelwert einer 0/1-Spalte ist der Anteil der Einsen, also eine Rate.

4.6 Darstellen mit Matplotlib

Die Überlebensraten aus dem letzten Kapitel lassen sich als Diagramm schneller vergleichen als in der Tabelle. Matplotlib ist die Bibliothek hierfür; bei der Kosinus-Kurve in LE01 hatten wir sie schon einmal verwendet. Das Modell dahinter hat zwei Teile: Die `Figure` ist das Blatt, die `Axes` sind ein Koordinatensystem darauf. `plt.subplots()` liefert beide. Gezeichnet und beschriftet wird mit Methoden des Koordinatensystems `ax`, etwa `ax.bar` oder `ax.set_title`; das Blatt `fig` braucht man zum Speichern und für die Anordnung mehrerer Koordinatensysteme. Diese Form ist im Kurs die einzige, die wir benötigen. Sie funktioniert auch mit mehreren Koordinatensystemen auf einem Blatt.

fig, ax = plt.subplots(): das Blatt und das Koordinatensystem darauf



`ax.bar(x, heights)`
zeichnet in dieses Koordinatensystem

`ax.set_xlabel(...)`
Beschriftung der x-Achse

`ax.set_ylabel(...)`
Beschriftung der y-Achse

`ax.set_title(...)`
Titel des Koordinatensystems

`fig.savefig(...)`
speichert das ganze Blatt

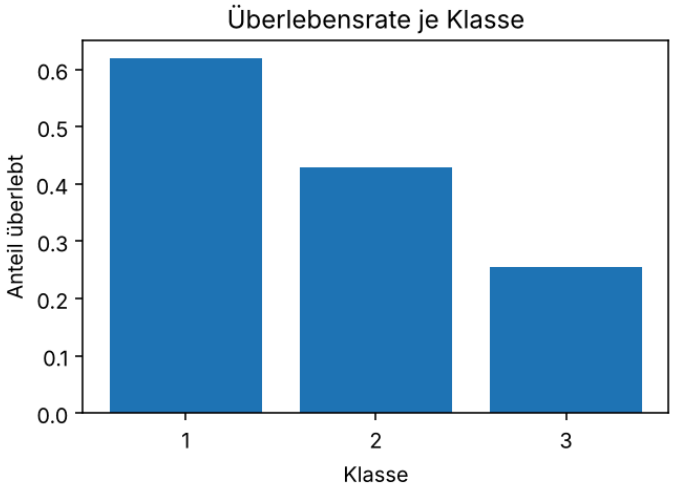
`plt.subplots(1, 3)`
ein Blatt mit drei Koordinatensystemen

Gezeichnet und beschriftet wird am Koordinatensystem ax. Das Blatt fig legt die Größe fest und wird als Ganzes gespeichert.

Ein Balkendiagramm für die Überlebensrate je Klasse: Die Werte kommen aus `groupby`, die Kategorien aus dem Index des Ergebnisses. Jede Achse bekommt eine Beschriftung, das Diagramm einen Titel. Das ist die Mindestausstattung, die Legende kommt beim Streudiagramm dazu.

```
   rates = clean.groupby("pclass")["survived"].mean()
① fig, ax = plt.subplots(figsize=(5, 3.2))
② ax.bar(rates.index.astype(str), rates.to_numpy())
③ ax.set_xlabel("Klasse")
   ax.set_ylabel("Anteil überlebt")
   ax.set_title("Überlebensrate je Klasse")
   plt.show()
```

① `plt.subplots()` liefert das Blatt `fig` und ein Koordinatensystem `ax`; `figsize` ist die Größe in Zoll
② `ax.bar(x, höhen)` zeichnet je Eintrag einen Balken: `x` sind die Klassen aus dem Index von `rates`, als Text, damit sie Kategorien bleiben; die Höhen sind die drei Überlebensraten aus `rates`, als Array
③ die drei Beschriftungen, jede eine Methode am `ax`



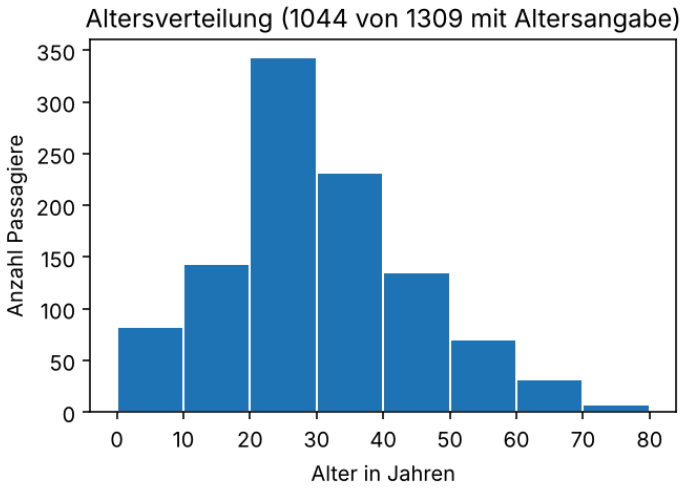
Ein Histogramm zeigt die Verteilung einer Zahlenspalte: Es teilt den Wertebereich in Intervalle und zählt je Intervall. Die Intervallgrenzen geben wir fest vor (`bins=range(0, 90, 10)`, also 0 bis 10, 10 bis 20 und so weiter bis 80), damit das Diagramm reproduzierbar ist. Fehlende Werte kann ein Histogramm nicht einsortieren. `dropna()` entfernt sie vorher, und die Anzahl der gezeichneten Werte gehört in den Titel, damit niemand 1309 Passagiere vermutet. Die Gesamttabelle bleibt dabei unverändert.

```
   AGE_BINS = range(0, 90, 10)
① ages = clean["age"].dropna()
   fig, ax = plt.subplots(figsize=(5, 3.2))
② ax.hist(ages, bins=AGE_BINS, edgecolor="white")
   ax.set_xlabel("Alter in Jahren")
   ax.set_ylabel("Anzahl Passagiere")
   ax.set_title(f"Altersverteilung ({len(ages)} von {len(clean)} mit Altersangabe)")
   plt.show()
   print("je Intervall:", np.histogram(ages, bins=AGE_BINS)[0].tolist())
```

① `dropna()` liefert die Series ohne fehlende Werte; `clean` selbst bleibt, wie sie ist

② `ax.hist(werte, bins=grenzen)` zählt je Intervall und zeichnet die Balken; `edgecolor` macht die Grenzen sichtbar

```
je Intervall: [82, 143, 343, 232, 135, 70, 32, 7]
```



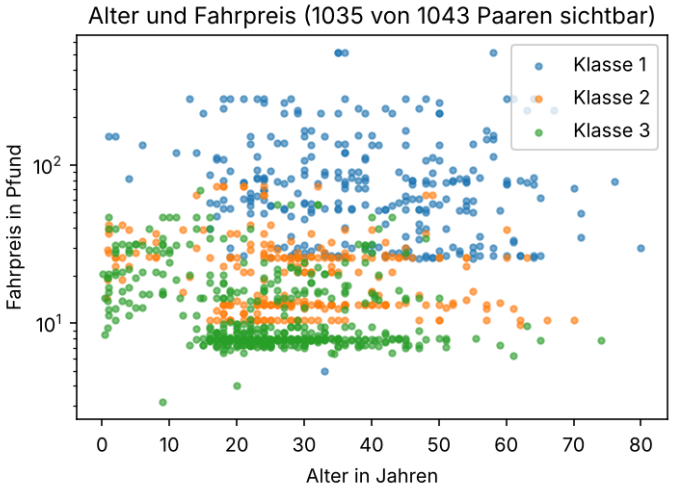
Ein Streudiagramm zeigt zwei Zahlenspalten gegeneinander, ein Punkt je Zeile: hier das Alter auf der x-Achse und der Fahrpreis auf der y-Achse. Dafür braucht jede Zeile beide Werte, und `dropna(subset=["age", "fare"])` behält nur die vollständigen Paare, 1043 von 1309.

Die Farbe nach Klasse entsteht in drei Schritten. Eine Schleife über `df.groupby("pclass")` liefert je Klasse den Schlüssel und die Teiltabelle; das ist der erste Schritt aus der Grafik Split-apply-combine, ohne das Zusammenfassen. Für jede Teiltabelle folgt ein eigener `scatter`-Aufruf. Das Argument `label` gibt jedem Aufruf seinen Eintrag in der Legende.

Der Fahrpreis ist schief verteilt: Die meisten Werte liegen unter 50 Pfund, wenige über 500. Auf einer gewöhnlichen Achse drängen sich die vielen kleinen Werte am unteren Rand. Eine logarithmische y-Achse (`ax.set_yscale("log")`) zieht sie auseinander: Gleiche Abstände bedeuten dort gleiche Faktoren, von 10 auf 100 ist so weit wie von 100 auf 1000. Der Preis dafür: Den Logarithmus von 0 gibt es nicht, und acht der 1043 Passagiere haben den Fahrpreis 0. Diese acht Punkte fehlen im Diagramm, deshalb nennt der Titel beide Zahlen.

```
① pairs = clean.dropna(subset=["age", "fare"])
  fig, ax = plt.subplots(figsize=(5.5, 3.6))
② for pclass, group in pairs.groupby("pclass"):
③     ax.scatter(group["age"], group["fare"], s=10, alpha=0.6, label=f"Klasse {pclass}")
  ax.set_xlabel("Alter in Jahren")
  ax.set_ylabel("Fahrpreis in Pfund")
  ax.set_title(f"Alter und Fahrpreis ({(pairs['fare'] > 0).sum()} von {len(pairs)} Paaren sichtbar)")
④ ax.set_yscale("log")
⑤ ax.legend()
  plt.show()
```

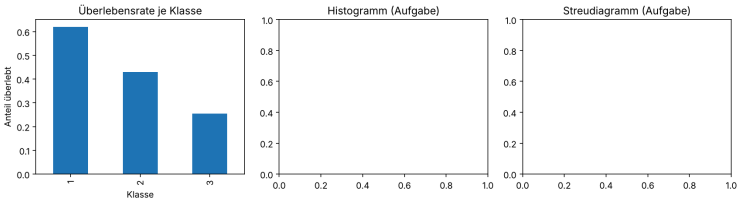
① `dropna(subset=[...])` entfernt nur Zeilen, in denen eine der genannten Spalten fehlt
② die Schleife über `groupby` liefert je Gruppe den Schlüssel (`pclass`) und die Teiltabelle (`group`)
③ ein `scatter` je Gruppe: `label` ist der Text in der Legende, `s` die Punktgröße, `alpha` die Deckkraft
④ `set_yscale("log")` stellt die y-Achse logarithmisch dar; Fahrpreis 0 fällt dabei weg (log(0) gibt es nicht)
⑤ `legend()` zeigt die Einträge aus den `label`-Angaben



Mehrere Koordinatensysteme auf einem Blatt liefert `plt.subplots(1, 3)`: ein Blatt, drei `Axes` nebeneinander, als Array aus drei `Axes`, das sich wie ein Tupel auspacken lässt. `df.plot(kind=..., ax=ax)` ist eine Abkürzung von pandas, die in ein gegebenes `ax` zeichnet. Sie liefert dasselbe `ax` zurück, Beschriftungen setzen Sie wie bisher. Die nächste Zelle zeigt beides und ist zugleich das Gerüst für die Aufgabe.

```
① fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 3.4))
② rates.plot(kind="bar", ax=ax1)
  ax1.set_xlabel("Klasse")
  ax1.set_ylabel("Anteil überlebt")
  ax1.set_title("Überlebensrate je Klasse")
  ax2.set_title("Histogramm (Aufgabe)")
  ax3.set_title("Streudiagramm (Aufgabe)")
③ fig.tight_layout()
  plt.show()
```

① `subplots(1, 3)` liefert ein Blatt und ein Array aus drei Koordinatensystemen, hier in drei Namen ausgepackt; `figsize` gilt für das ganze Blatt
② `series.plot(kind="bar", ax=ax1)` zeichnet die Series als Balken in `ax1`
③ `tight_layout()` rückt die Koordinatensysteme so, dass Beschriftungen nicht überlappen



Vertiefung: Diagramme speichern

NICHT PRÜFUNGSRELEVANT

`fig.savefig("analyse.png", dpi=150)` schreibt das Blatt als Bilddatei. `dpi` ist die Auflösung in Punkten je Zoll, 150 reicht für Dokumente, 300 für den Druck.

Das Format folgt der Dateiendung: `.png` für Rasterbilder aus Bildpunkten (Pixeln), `.svg` oder `.pdf` für Vektorgrafiken, die sich verlustfrei skalieren lassen. `bbox_inches="tight"` schneidet überflüssigen Rand ab. In Notebooks zeigt `plt.show()` das Blatt an. Wenn Sie beides wollen, rufen Sie `savefig` am einfachsten vor `show` auf. Mit der Referenz `fig` funktioniert `fig.savefig` auch danach. Eine bekannte Fehlerquelle ist `plt.savefig()` nach einem blockierenden `plt.show()` in einem Skript außerhalb des Notebooks, dort ist das aktuelle Blatt dann schon geschlossen.

✏️ Aufgabe 4.6 #1

10 min

Die Aufgabe hat zwei Teile: Zuerst schreiben Sie zwei Datenfunktionen, die eine eigene Prüfzelle testet, danach zeichnen Sie.

1. Schreiben Sie `valid_ages(df)`, das die Alterswerte ohne Lücken als Series zurückgibt, und `age_fare_pairs(df)`, das die Zeilen mit Alter und Fahrpreis als DataFrame mit genau den Spalten `age`, `fare` und `pclass` zurückgibt. Die Prüfzelle prüft Anzahl und Werte.
2. Zeichnen Sie in der Zelle danach in das vorgegebene Gerüst: in `ax2` das Histogramm von `valid_ages(clean)` mit den Intervallgrenzen `AGE_BINS`, in `ax3` das Streudiagramm Alter gegen Fahrpreis aus `age_fare_pairs(clean)`, ein `scatter` je Klasse mit `label`, logarithmischer y-Achse und Legende. Beide mit Achsenbeschriftung und, wie in den Demo-Zellen, mit der Anzahl der dargestellten Werte im Titel; bei drei Diagrammen nebeneinander passt die Anzahl in eine zweite Titelzeile (`\n` im Titeltext). Die Prüfzelle prüft die Intervalle und Balkenhöhen, die Punkte je Klasse und die Beschriftung.

Wer früher fertig ist: Schreiben Sie in einer neuen Zelle `age_by_sex(df)`, das ein Dict mit den Alterswerten je Geschlecht liefert (`{"female": Series, "male": Series}`), und zeichnen Sie beide als Histogramme in ein gemeinsames `ax` mit `alpha=0.5` und `label`, damit sich die Verteilungen überlagern.

Hinweis: Beide Datenfunktionen stehen fast wörtlich in den Zellen oben (`dropna()` und `dropna(subset=[...])`), nur die Spaltenauswahl `[["age", "fare", "pclass"]]` kommt dazu. Für das Zeichnen übernehmen Sie die Zeilen aus den Demo-Zellen und ersetzen `ax` durch `ax2` beziehungsweise `ax3`.

```
def valid_ages(df):
    # TODO: Ihre Lösung hier
    ...

def age_fare_pairs(df):
    # TODO: Ihre Lösung hier
    ...
```

```
    # Prüfzelle – einfach ausführen
    ages = valid_ages(clean)
    pairs = age_fare_pairs(clean)
    assert ages is not None and pairs is not None, "eine der beiden Funktionen gibt noch nichts zurück"
    assert len(ages) == 1044 and ages.isna().sum() == 0, f"1044 Alterswerte ohne Lücken erwartet, nicht {len(ages)}"
    # Vergleichswerte unabhängig über Masken gebildet; die Reihenfolge ist frei, verglichen wird sortiert
    ref_ages = clean.loc[clean["age"].notna(), "age"].to_numpy()
    assert np.array_equal(np.sort(ages.to_numpy()), np.sort(ref_ages)), "valid_ages soll genau die vorhandenen Alterswerte liefern, jeden einmal"
    assert list(pairs.columns) == ["age", "fare", "pclass"], "age_fare_pairs: genau die Spalten age, fare, pclass"
    assert len(pairs) == 1043 and pairs.isna().sum().sum() == 0, f"1043 vollständige Paare erwartet, nicht {len(pairs)}"
    ref_pairs = clean.loc[clean["age"].notna() & clean["fare"].notna(), ["age", "fare", "pclass"]]
    assert np.array_equal(pairs.sort_values(["age", "fare", "pclass"]).to_numpy(),
                          ref_pairs.sort_values(["age", "fare", "pclass"]).to_numpy()), "age_fare_pairs: die Werte einer Zeile müssen zusammenbleiben (zeilenweise auswählen, nicht spaltenweise)"
    print("Alle Prüfungen bestanden ✓")
```

```
    # Gerüst: das Blatt mit drei Koordinatensystemen, ax1 ist fertig, ax2 und ax3 sind Ihre Aufgabe
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 3.4))
    rates.plot(kind="bar", ax=ax1)
    ax1.set_xlabel("Klasse")
    ax1.set_ylabel("Anteil überlebt")
    ax1.set_title("Überlebensrate je Klasse")
    # TODO: Ihre Lösung hier
    ...
    fig.tight_layout()
    plt.show()
```

```
   # Prüfzelle – einfach ausführen
   # Vergleichswerte unabhängig über Masken, wie in der Prüfzelle davor
   ref_ages = clean.loc[clean["age"].notna(), "age"].to_numpy()
   ref_pairs = clean.loc[clean["age"].notna() & clean["fare"].notna(), ["age", "fare", "pclass"]]
   # Histogramm: ein Balken je Intervall, Lage und Breite aus AGE_BINS, Höhen = Anzahl je Intervall
① assert len(ax2.patches) == 8, f"acht Balken für die Intervalle 0 bis 80 erwartet, nicht {len(ax2.patches)}"
   assert [round(p.get_x()) for p in ax2.patches] == list(AGE_BINS)[:-1], "die Balken sollen bei 0, 10, …, 70
beginnen (bins=AGE_BINS)"
   assert all(round(p.get_width()) == 10 for p in ax2.patches), "jeder Balken soll zehn Jahre breit sein"
   heights = [p.get_height() for p in ax2.patches]
   expected = np.histogram(ref_ages, bins=AGE_BINS)[0].tolist()
   assert heights == expected, f"Balkenhöhen {heights} passen nicht zu den Häufigkeiten {expected}"
   assert ax2.get_xlabel() and ax2.get_ylabel(), "ax2: beide Achsen beschriften"
   assert str(len(ref_ages)) in ax2.get_title(), f"ax2: Titel mit der Anzahl der dargestellten Werte
({len(ref_ages)})"
   # Streudiagramm: je Legendeneintrag die Punkte der passenden Klasse, Mehrfachpunkte mitgezählt
   assert len(ax3.collections) == 3, f"ein scatter je Klasse, also drei, nicht {len(ax3.collections)}"
   assert ax3.get_legend() is not None, "ax3: Legende fehlt"
② handles, labels = ax3.get_legend_handles_labels()
   assert len(set(labels)) == 3 and all(any(ch in lab for ch in "123") for lab in labels), "ax3: drei
Legendeneinträge mit der Klassennummer, zum Beispiel 'Klasse 1'"
   def sorted_rows(a):                                  # Zeilen eines (N, 2)-Arrays lexikografisch ordnen,
für den Vergleich zweier Punktmengen
       a = np.asarray(a, dtype=float).round(4)
       return a[np.lexsort((a[:, 1], a[:, 0]))]
   total = 0
   for handle, lab in zip(handles, labels):
       pclass = int(next(ch for ch in lab if ch in "123"))
       drawn = np.asarray(handle.get_offsets(), dtype=float)
       drawn = drawn[np.isfinite(drawn).all(axis=1)]
       wanted = ref_pairs.loc[ref_pairs["pclass"] == pclass, ["age", "fare"]].to_numpy()
       assert len(drawn) == len(wanted), f"'{lab}': {len(wanted)} Punkte erwartet, gezeichnet {len(drawn)};
stimmt die Zuordnung von Gruppe und label?"
       assert np.array_equal(sorted_rows(drawn), sorted_rows(wanted)), f"'{lab}': die Punkte sollen (Alter,
Fahrpreis) der Klasse {pclass} sein; sind die Achsen vertauscht oder die Klassen verwechselt?"
       total += len(drawn)
   assert total == 1043, f"1043 Punkte erwartet, nicht {total}"
   assert len({tuple(c.get_facecolor()[0]) for c in ax3.collections}) == 3, "ax3: drei verschiedene Farben,
eine je Klasse"
   assert ax3.get_yscale() == "log", "ax3: y-Achse logarithmisch"
   assert ax3.get_xlabel() and ax3.get_ylabel(), "ax3: beide Achsen beschriften"
   assert str(int((ref_pairs["fare"] > 0).sum())) in ax3.get_title(), f"ax3: Titel mit der Anzahl der
sichtbaren Punkte ({int((ref_pairs['fare'] > 0).sum())})"
   print("Alle Prüfungen bestanden ✓")
```

① `ax.patches` sind die gezeichneten Rechtecke; `get_x()`, `get_width()` und `get_height()` ihre Lage und Größe

② `get_legend_handles_labels()` liefert die gezeichneten Punktmengen und ihre `label`-Texte paarweise; `get_offsets()` sind die Koordinaten als Array (N, 2)

Überblick: Figure und Axes

Aufruf	Wirkung
`fig, ax = plt.subplots(figsize=(w, h))`	ein Blatt mit einem Koordinatensystem
`fig, (ax1, ax2) = plt.subplots(1, 2)`	ein Blatt, zwei Koordinatensysteme nebeneinander
`ax.bar(kategorien, höhen)`	Balken
`ax.hist(werte, bins=grenzen)`	Histogramm, Werte vorher ohne Lücken
`ax.scatter(x, y, label=...)`	Punkte, ein Aufruf je Gruppe für die Legende
`ax.set_xlabel`, `set_ylabel`, `set_title`	Beschriftung
`ax.legend()`, `ax.set_yscale("log")`	Legende, logarithmische Achse
`series.plot(kind="bar", ax=ax)`	pandas-Abkürzung, zeichnet in das gegebene `ax`
`fig.tight_layout()`, `fig.savefig(...)`	Anordnung, Speichern

Ein Diagramm braucht Achsenbeschriftung, Titel und, bei mehreren Gruppen, eine Legende. Die Anzahl der dargestellten Werte gehört in den Titel, wenn Lücken weggelassen wurden.

4.7 Von der Tabelle zum Array

Am Ende des Datenworkflows steht wieder ein Array, denn das Modell in der übernächsten Lerneinheit rechnet auf Tensoren, und die entstehen aus Arrays. Zwei Festlegungen gehören dazu: welche Spalten die Merkmale sind, und dass die Zielspalte getrennt bleibt. Die Merkmalsspalten für dieses Beispiel sind die vier Zahlenspalten `age`, `sibsp`, `parch` und `fare`. `pclass` ist eine Kategorie und bleibt hier draußen, obwohl Zahlen darin stehen (die nächste Lerneinheit macht aus Kategorien Zahlen, die ein Modell verwenden kann). Das scikit-learn-Beispiel nimmt nur `age` und `fare` als Zahlen. Die vier Spalten hier zeigen nur den Übergang zum Array.

💭 Kurz überlegen

`clean[num_cols].to_numpy(dtype="float32")` mit den vier Merkmalsspalten: Welche Form und welchen Datentyp hat das Array?

```
   num_cols = ["age", "sibsp", "parch", "fare"]
① X = clean[num_cols].to_numpy(dtype="float32")
   y = clean["survived"].to_numpy()
   print("X:", X.shape, X.dtype, "| fehlende Werte:", int(np.isnan(X).sum()))
   print("y:", y.shape, y.dtype, "| Anteil überlebt:", y.mean().round(3))
② print("Spalte 0 ist", num_cols[0], "-> erste Zeile:", X[0])
   # Formen: clean (1309, 11) -> X (1309, 4);  clean (1309, 11) -> y (1309,)
```

① `to_numpy(dtype="float32")` liefert die Werte der gewählten Spalten als Array mit diesem Datentyp
② die Spaltennamen gehen beim Übergang verloren; `num_cols` hält fest, welche Spalte welche Achse-1-Position hat

```
X: (1309, 4) float32 | fehlende Werte: 266
y: (1309,) int64 | Anteil überlebt: 0.382
Spalte 0 ist age -> erste Zeile: [ 29.       0.       0.     211.3375]
```

Was der Übergang garantiert und was nicht: Form, Bedeutung der Achsen und Datentyp sind festgelegt. Nicht festgelegt ist die Speicheranordnung. `to_numpy` darf das Array spaltenweise ablegen, und dann ist es nicht zeilenweise zusammenhängend, so wie `E.T` in LE03. Für pandas und NumPy spielt das keine Rolle. Verlangt ein Vertrag wie der von `row_l2_normalize` die zeilenweise Anordnung, stellt `np.ascontiguousarray` sie her.

Vertiefung: Speicheranordnung und Schreibschutz nach to_numpy

Zwei Eigenschaften des Arrays aus `to_numpy` sind nicht garantiert, und beide kommen beim Übergang zum Tensor als Kernstoff wieder.

Die erste ist die Speicheranordnung. `X.flags["C_CONTIGUOUS"]` sagt, ob die Zeilen dicht hintereinander im Speicher liegen (zeilenweise Anordnung, wie in LE03). Nach `to_numpy` auf mehreren Spalten ist das oft nicht der Fall, weil pandas die Spalten getrennt speichert und das Array spaltenweise aufbaut. `np.ascontiguousarray(X)` kopiert in die zeilenweise Anordnung, wenn nötig, und gibt sonst `X` selbst zurück. PyTorch arbeitet mit beiden Anordnungen, aber eigene Kernel und der Semester-Operator setzen die zeilenweise Anordnung voraus.

Die zweite ist der Schreibschutz. Unter Copy-on-Write liefert `to_numpy()` für eine einzelne Spalte eine schreibgeschützte Sicht auf die Daten der Tabelle: `y.flags["WRITEABLE"]` ist `False`, und `y[0] = 1` bricht ab, damit niemand an der Tabelle vorbei in ihre Daten schreibt. Ob ein mehrspaltiges Array eine Kopie ist, hängt von den Spalten und der Umwandlung ab; `X` mit `dtype="float32"` ist hier eine Kopie, weil die Umwandlung in einen anderen Datentyp kopieren muss. Wer das Array verändern will, verlässt sich nicht darauf, sondern holt mit `to_numpy(copy=True)` eine eigene, beschreibbare Kopie. `np.ascontiguousarray` hebt den Schreibschutz nicht auf, wenn es nicht kopieren muss.

```
   print("zeilenweise zusammenhängend:", X.flags["C_CONTIGUOUS"])
   Xc = np.ascontiguousarray(X)
   print("nach ascontiguousarray:", Xc.flags["C_CONTIGUOUS"], "| gleiche Werte:", np.array_equal(X, Xc,
equal_nan=True))
   print("beschreibbar: X", X.flags["WRITEABLE"], "| y", y.flags["WRITEABLE"],
         "| y mit copy=True", clean["survived"].to_numpy(copy=True).flags["WRITEABLE"])
```

```
zeilenweise zusammenhängend: False
nach ascontiguousarray: True | gleiche Werte: True
beschreibbar: X True | y False | y mit copy=True True
```

Zum Schluss wird die bereinigte Tabelle gespeichert. `to_csv(index=False)` schreibt sie als CSV-Datei ohne die Index-Spalte. Die nächste Lerneinheit liest genau diese Datei wieder ein. Wie Dateien im Einzelnen geschrieben und gelesen werden, behandelt Grundkurs Teil 3.

```
① clean.to_csv("data/titanic_clean.csv", index=False)
   check = pd.read_csv("data/titanic_clean.csv")
   print(check.shape, "| gleich wie clean:", check.equals(clean))
```

① `to_csv(pfad, index=False)` schreibt die Tabelle; ohne `index=False` käme der Index als erste Spalte mit in die Datei

```
(1309, 11) | gleich wie clean: True
```

✏️ Aufgabe 4.7 #1 5 min

Hausaufgabe, falls im Termin keine Zeit bleibt. Schreiben Sie `to_arrays(df, feature_cols)`, das aus einer bereinigten Tabelle das Merkmalsarray `X` (Form `(N, D)`, `float32`) und die Zielspalte `y` (Form `(N,)`, die Werte von `survived`) als Tupel zurückgibt. Die Prüfzelle prüft Form, Datentyp, Wertebereich und dass `survived` nicht in `X` steckt.

Hinweis: `feature_cols` übernimmt die Rolle von `num_cols`, `survived` kommt getrennt über `to_numpy()`.

```
   def to_arrays(df, feature_cols):
       # TODO: Ihre Lösung hier
       ...
```

```
    # Prüfzelle – einfach ausführen
    result = to_arrays(clean, num_cols)
    assert result is not None, "to_arrays gibt noch nichts zurück (return vergessen?)"
    X, y = result
    assert X.shape == (1309, 4) and X.dtype == np.float32, f"X: Form (1309, 4) und float32 erwartet, nicht
{X.shape} {X.dtype}"
    assert y.shape == (1309,) and set(np.unique(y)) == {0, 1}, "y: Form (1309,) mit den Werten 0 und 1"
    assert np.nanmin(X[:, 0]) >= 0 and np.nanmax(X[:, 0]) <= 100, "Spalte 0 ist age, zwischen 0 und 100"
    assert not any(np.array_equal(np.nan_to_num(X[:, j]), y) for j in range(X.shape[1])), "survived darf nicht
in X stecken"
    assert np.isnan(X).sum() == 266, "die 265 fehlenden Alter und der eine fehlende Fahrpreis bleiben NaN"
    # Werte und Reihenfolge: Spalte j von X ist feature_cols[j], Zeile i gehört zu Zeile i der Tabelle, y
dieselben Zeilen
    for j, col in enumerate(num_cols):
        assert np.allclose(X[:, j], clean[col].to_numpy(dtype="float32"), equal_nan=True), f"Spalte {j} von X
soll {col} sein (Reihenfolge von feature_cols)"
    assert np.array_equal(y, clean["survived"].to_numpy()), "y soll die Werte von survived in der
Zeilenreihenfolge der Tabelle sein"
    X2, y2 = to_arrays(clean.head(10), ["fare", "age"])              # andere Reihenfolge, anderer Ausschnitt
    assert X2.shape == (10, 2) and y2.shape == (10,), "feature_cols bestimmt die Spalten: zwei Spalten ergeben
(10, 2), y hat (10,)"
    assert np.allclose(X2[:, 0], clean["fare"].head(10), equal_nan=True) and np.allclose(X2[:, 1],
clean["age"].head(10), equal_nan=True), "X2: erst fare, dann age, wie in feature_cols übergeben"
    assert np.array_equal(y2, clean["survived"].head(10).to_numpy()), "y2: survived der ersten zehn Zeilen"
    print("Alle Prüfungen bestanden ✓")
```

Überblick: Von der Tabelle zum Array

Ausdruck	Ergebnis
`df[cols].to_numpy(dtype="float32")`	Array `(N, D)`, Spalten in der Reihenfolge von `cols`
`df["survived"].to_numpy()`	Array `(N,)`, die Zielspalte getrennt
`np.isnan(X).sum()`	Anzahl der Lücken im Array
`X.flags["C_CONTIGUOUS"]`, `np.ascontiguousarray(X)`	Speicheranordnung prüfen und herstellen
`X.flags["WRITEABLE"]`, `to_numpy(copy=True)`	Schreibschutz prüfen, beschreibbare Kopie holen
`df.to_csv(pfad, index=False)`	Tabelle als CSV speichern, ohne Index-Spalte

Was feststeht: Form, Bedeutung der Achsen, Datentyp. Was die Pipeline der nächsten Lerneinheit übernimmt: Lücken füllen, Kategorien in Zahlen wandeln, Merkmale skalieren.

Vertiefung: Mehrere Tabellen

NICHT PRÜFUNGSRELEVANT

Diese Lerneinheit arbeitet mit einer Tabelle. Sobald Daten aus mehreren Dateien kommen, braucht es zwei weitere Werkzeuge: `pd.concat` hängt Tabellen gleicher Spalten untereinander, `pd.merge` verbindet zwei Tabellen über eine gemeinsame Schlüsselspalte.

`pd.concat([januar, februar])` stapelt zum Beispiel Monatsdateien zu einer Tabelle. Die Etiketten der Teile bleiben erhalten und wiederholen sich, `ignore_index=True` vergibt neue. `pd.merge(passagiere, kabinen, on="ticket")` holt zu jeder Fahrkarte die Zusatzinformation aus der zweiten Tabelle, wie ein Join in SQL. Die Falle dabei: Kommt ein Schlüssel in der zweiten Tabelle mehrfach vor, vervielfacht `merge` die Zeilen der ersten. Prüfen Sie deshalb nach einem `merge` die Zeilenzahl gegen die Erwartung, so wie die Prüfzelle in diesem Notebook die 1309 Zeilen prüft.

Vertiefung: pandas oder Polars?

NICHT PRÜFUNGSRELEVANT

Polars ist eine jüngere Bibliothek für Tabellen, in Rust geschrieben, mit einer anderen Schreibweise und oft schnelleren Abfragen auf großen Tabellen. pandas ist die Bibliothek, die Lehrbücher, Beispiele und die meisten Projekte verwenden und die mit den Kurswerkzeugen gut zusammenspielt. scikit-learn und Matplotlib nehmen Arrays entgegen, und pandas liefert sie. Deshalb lernen wir sie hier.

Beide arbeiten mit Spalten über Arrays, und wer pandas versteht, liest Polars-Code nach kurzer Eingewöhnung. In Projekten entscheidet meist, mit welchen Bibliotheken die Daten weiterverarbeitet werden, seltener die Geschwindigkeit einzelner Abfragen.

➡️ Nächste Schritte

- Beenden Sie die Aufgaben, die im Termin offen geblieben sind, mindestens die Hausaufgabe `to_arrays`. Zur Schlusskontrolle starten Sie die Laufzeit neu und führen alle Zellen aus: Alle Prüfzellen sollten mit Ihren eigenen Lösungen durchlaufen.
- Grundkurs Teil 3 (Dateien, Fehler und Module) liegt in Moodle. Danach folgt die Projektaufgabe „Der Datenworkflow am zweiten Datensatz“: derselbe Weg wie in dieser Lerneinheit, an den Rohdaten einer Pinguin-Studie, mit eigenen Entscheidungen beim Bereinigen. Sie setzt Teil 3 voraus, nennt je Schritt das Kapitel, das er braucht, und kommt mit der nächsten E-Mail.
- Die Referenzkarte pandas fasst die Werkzeuge dieser Lerneinheit auf zwei Seiten zusammen, mit einer Karte zu Matplotlib als Vorgriff; die eigene Referenzkarte Matplotlib kommt mit LE06.

→ **Ausblick:** In der nächsten Lerneinheit wird aus der bereinigten Tabelle die Eingabe für ein Modell: eine scikit-learn-Pipeline füllt die Lücken, wandelt die Kategorien in Zahlen und übergibt das Ergebnis als Tensor an PyTorch.

Quellen und Referenzen

- W. McKinney: *Python for Data Analysis*, 3. Auflage, Kapitel 5 (pandas-Grundlagen), 6.1 (`read_csv`), 7 (Datenbereinigung), 9.1 (Matplotlib) und 10 (Gruppieren), wesmckinney.com/book.
- J. VanderPlas: *Python Data Science Handbook*, 2. Auflage, Kapitel 14 (Indexing and Selection) und 16 (Missing Data), jakevdp.github.io/PythonDataScienceHandbook.
- pandas-Dokumentation: Copy-on-Write, Indexing and selecting data, Group by, Migration guide for the new string data type.
- Matplotlib-Dokumentation: Quick start guide.
- scikit-learn-Dokumentation: Column Transformer with Mixed Types, das Beispiel mit dem Titanic-Datensatz, das die nächste Lerneinheit als Vorlage nimmt.
- Datengrundlage: Titanic-Passagierliste, F. E. Harrell Jr. und T. Cason, Vanderbilt University Department of Biostatistics, nach der Encyclopedia Titanica; auf OpenML als Datensatz 40945, Lizenz Public. Die Kursfassung `titanic_raw.csv` ist daraus abgeleitet, die Eingriffe stehen im Datenblatt `DATENBLATT_titanic.md`. Absätze der Suchdaten: GermanQuAD (deepset), CC BY 4.0, Texte aus der deutschen Wikipedia (CC BY-SA).

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT, Karikaturen: Google Gemini). Alle Inhalte sind von Prof. Dr. Stephan Kurpjuweit konzipiert, geprüft und verantwortet.